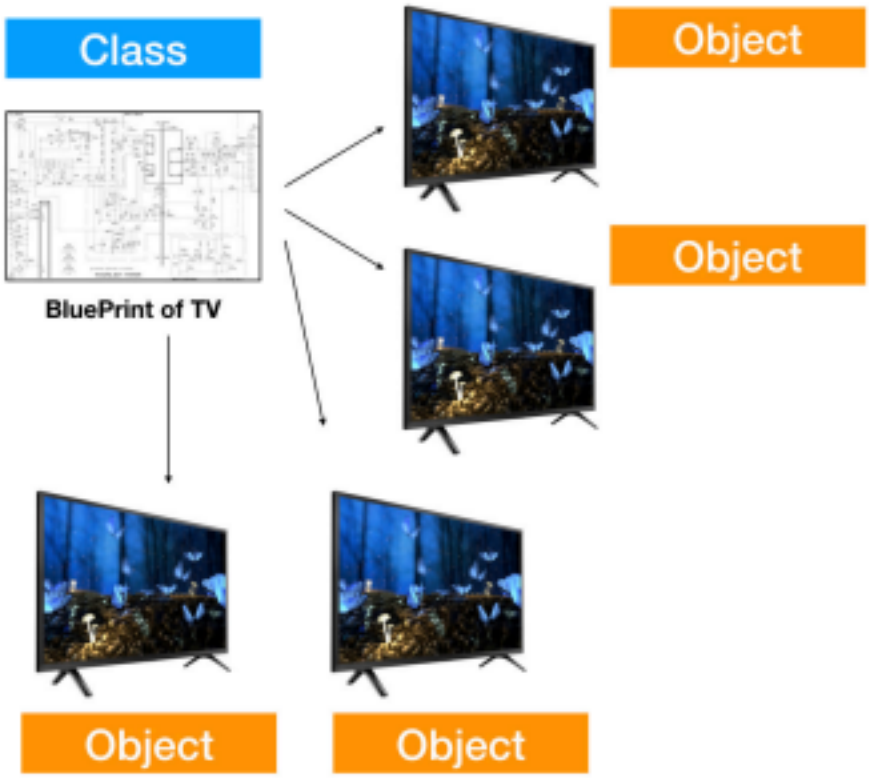
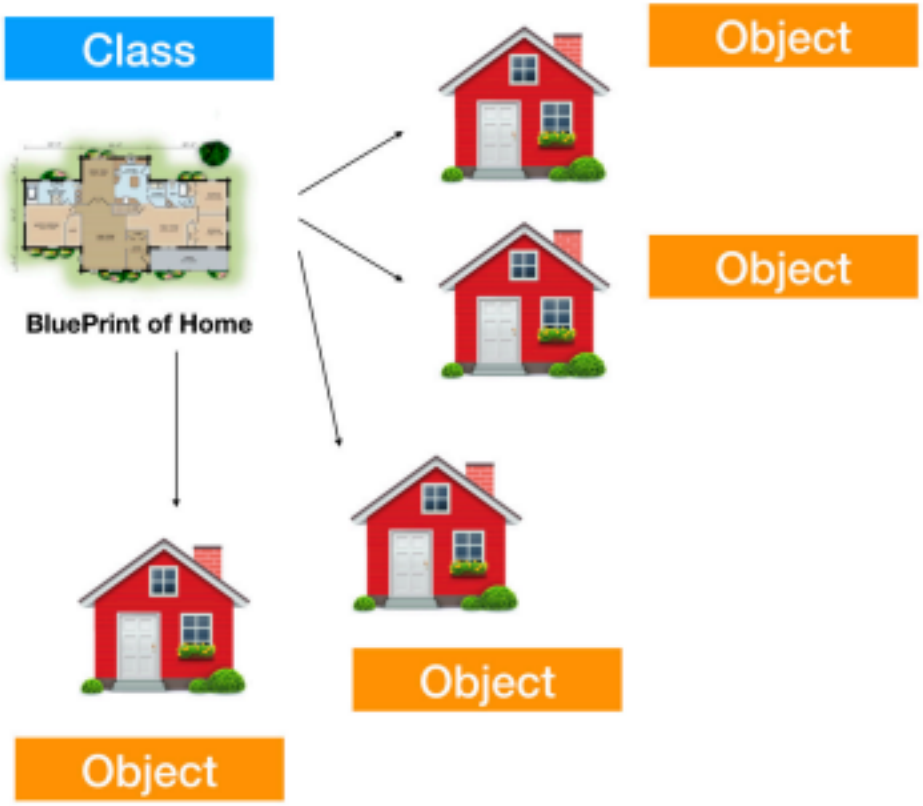
OOP(Object Oriented Programming)
Concept Created by: Alan Kay

First OOP based programming: Simula
First famous OOP based programming:
C++ Java's OOP comes from C++
Java is more Object Oriented and Secure than C++ Java is C++ without the guns, clubs and knives. (by- James Gosling)

Class is a blueprint of the object.

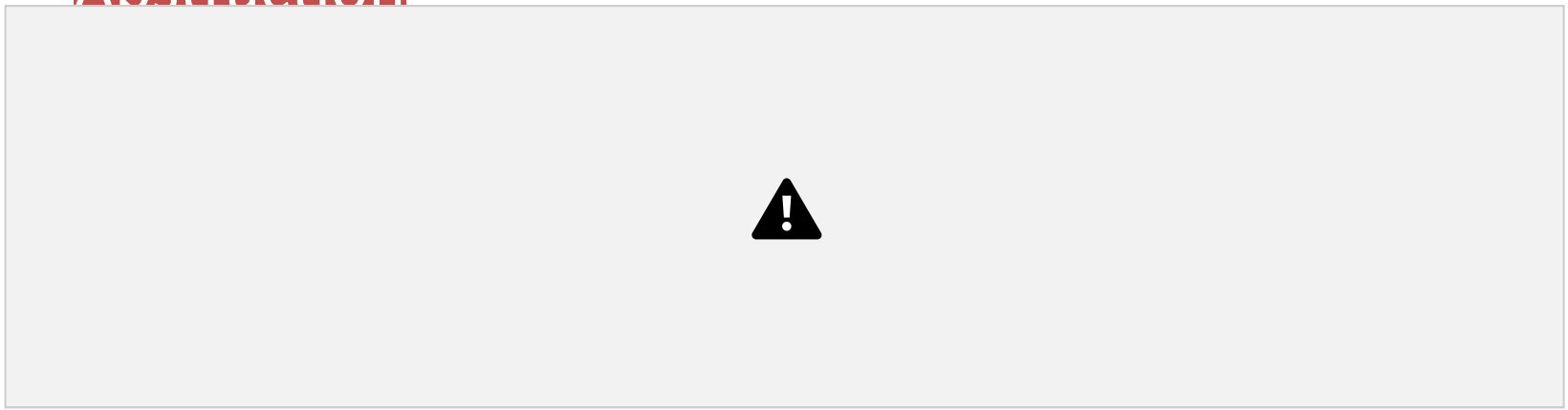
Object is a physical entity.




Class
Object
Object
BluePrint of Home
Object
Object

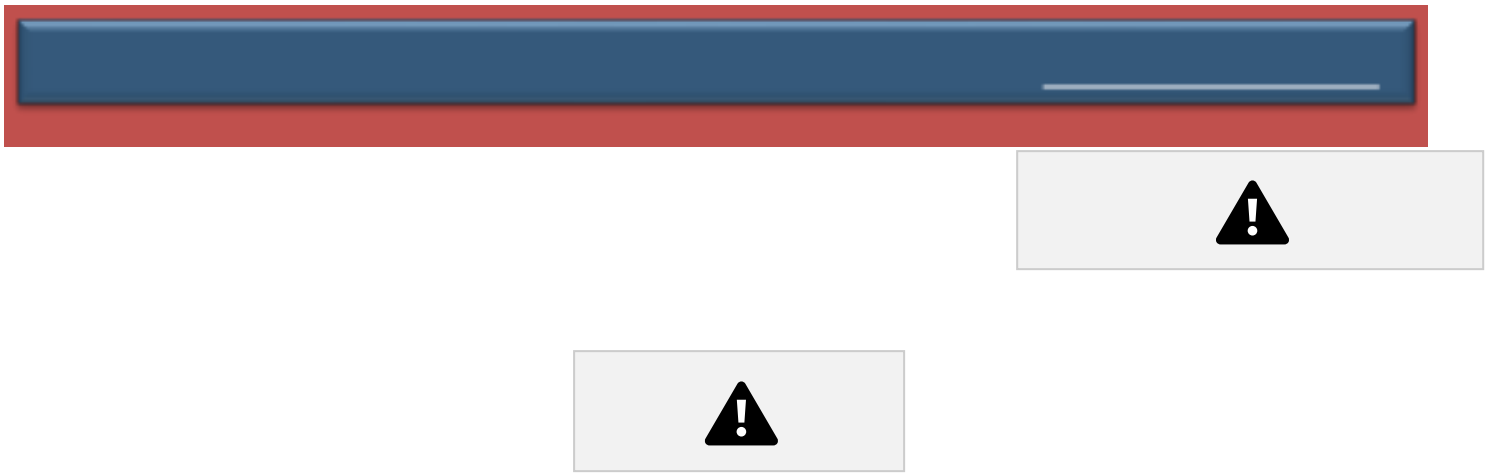
Encapsulation

Abstraction



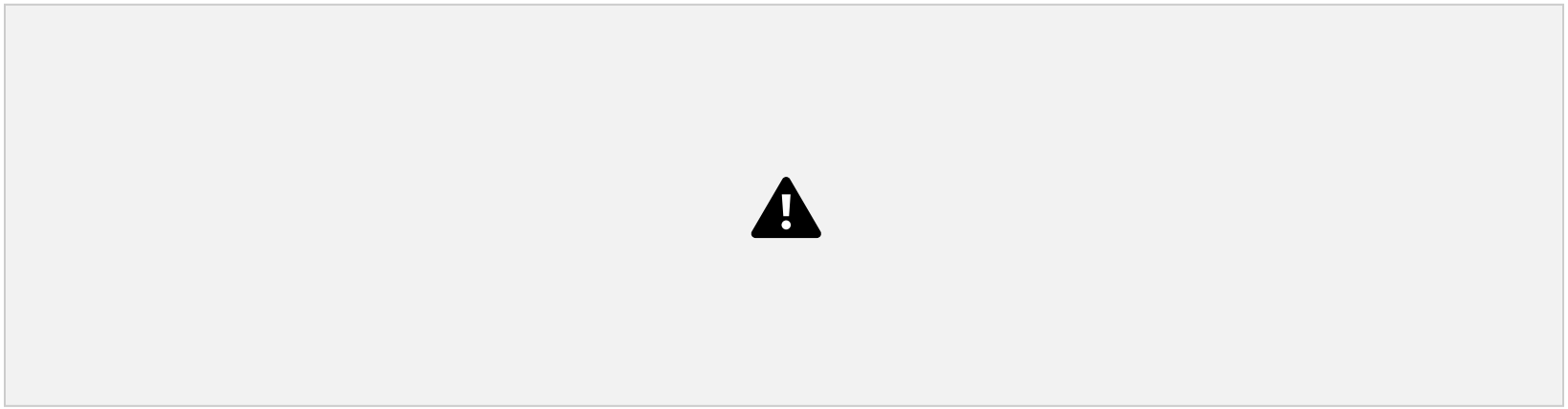
Inheritance

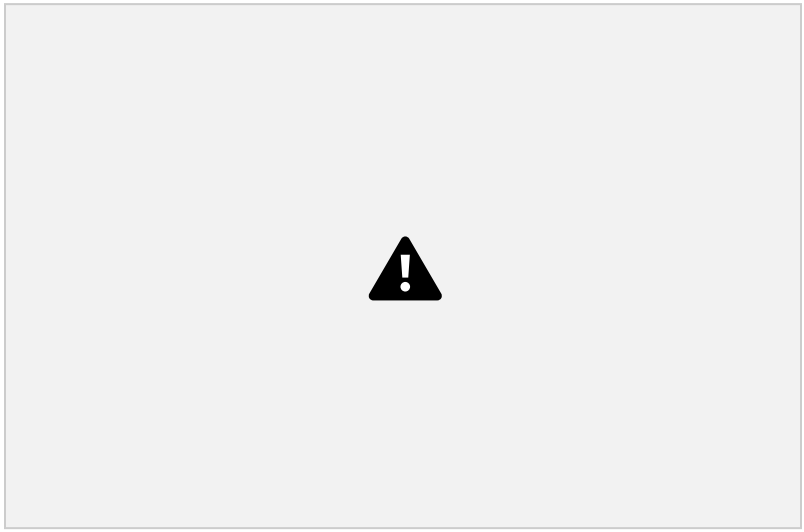
Polymorphism



Binding/Wrapping the component together to make them secure is known as **Encapsulation.**

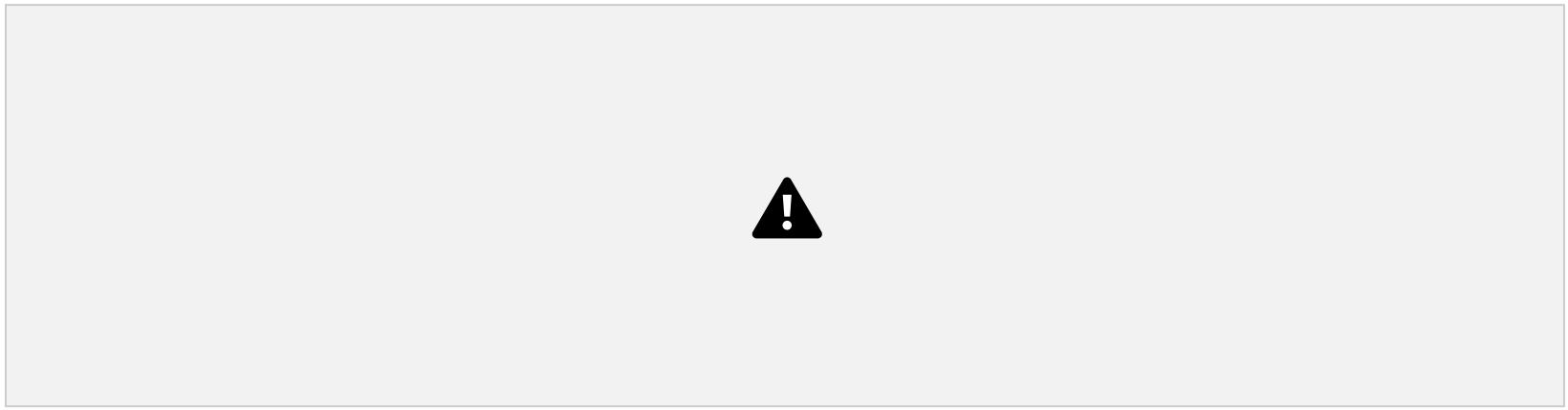
Hiding the complex working(Complexity) by providing the functionality is known as **Abstraction.**



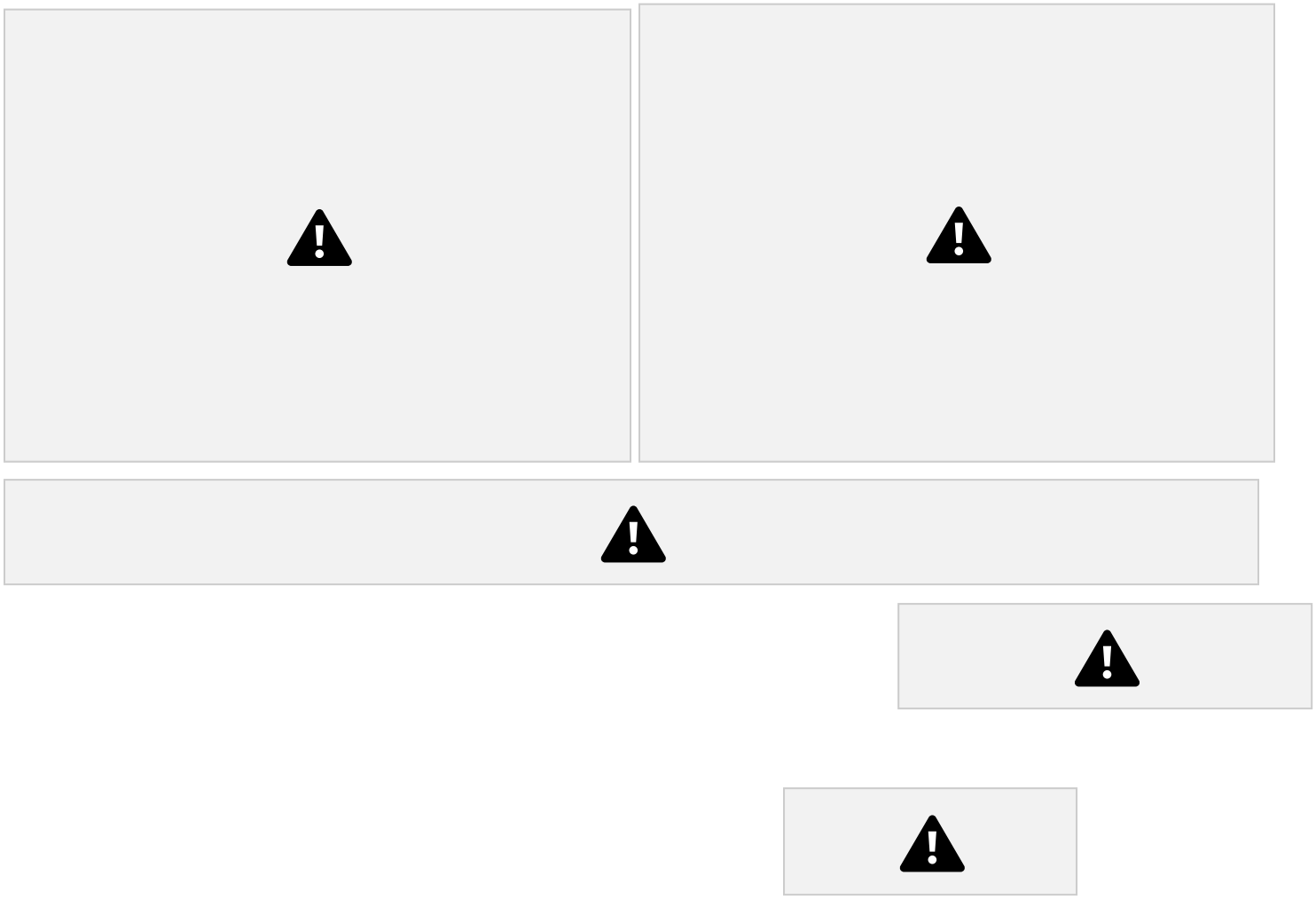


Using the old design to creating new design is known as **Inheritance.**

Many forms of doing a job.

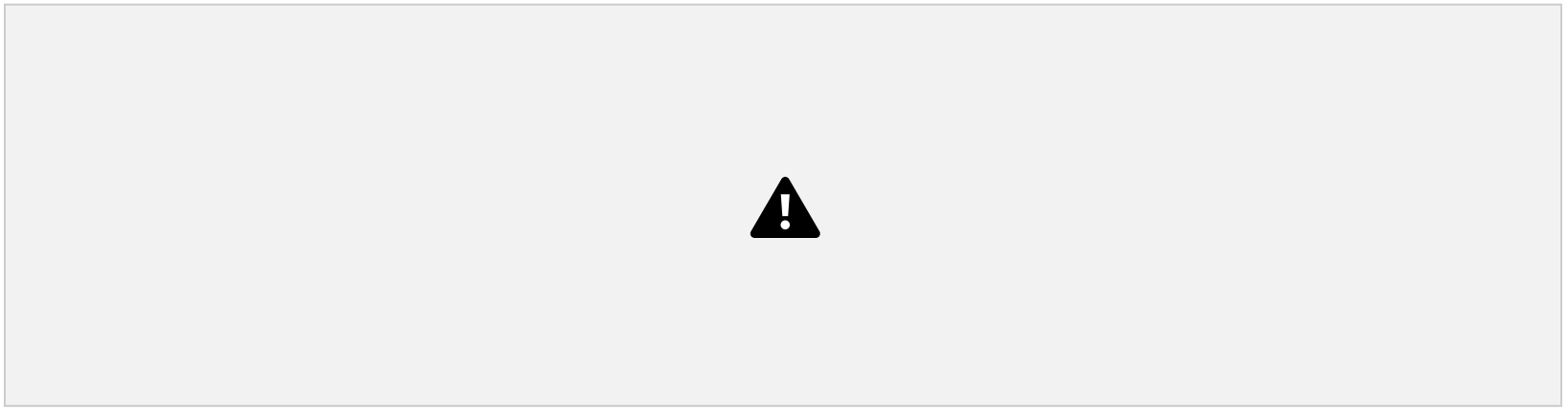


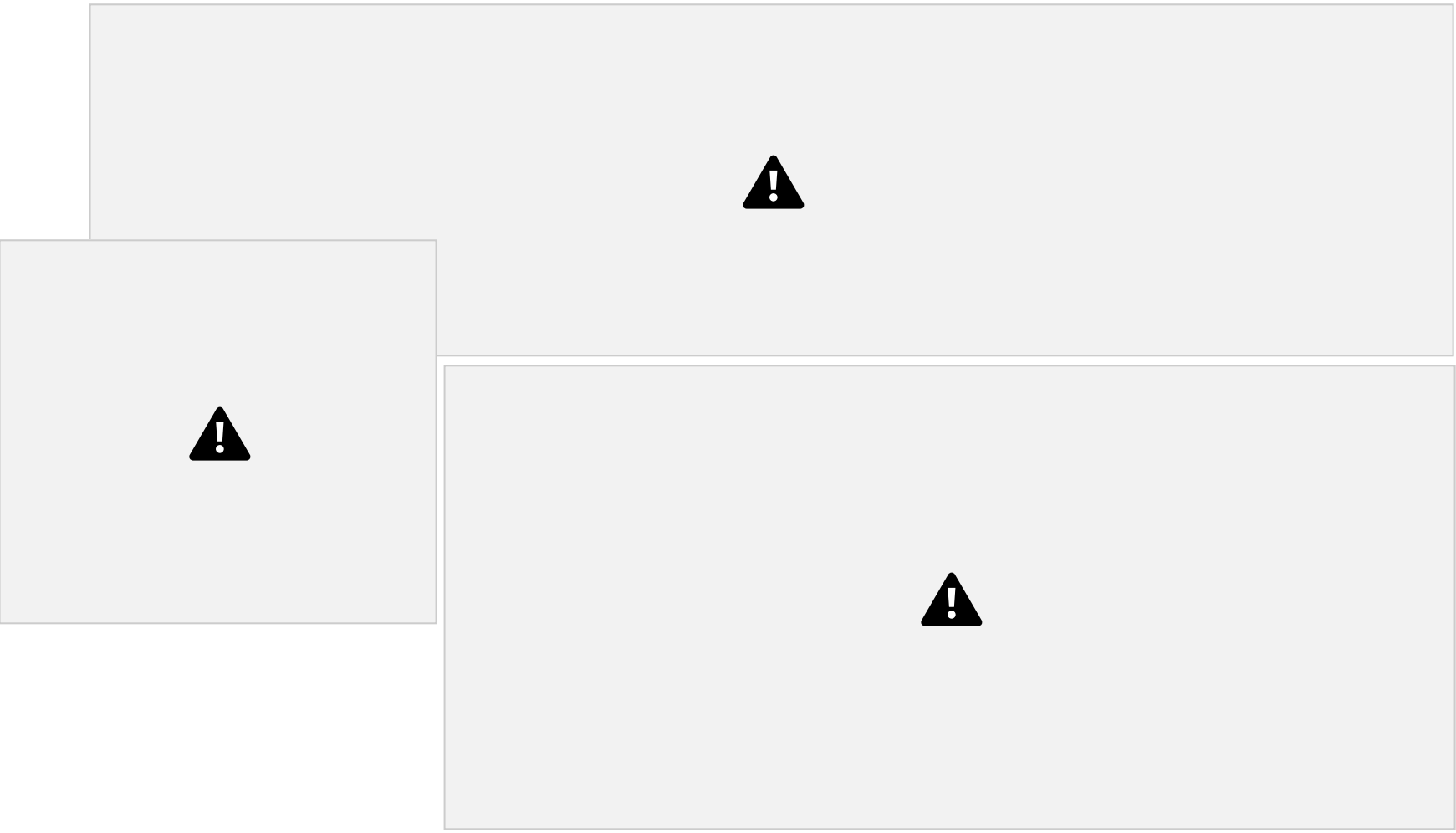
Examples- 1) Display has many forms. 2) Fly has many forms.



- **Only one class can be public in a java program.**
- **If a class is public in a java program then Java program name must**

be same as name of that public class.



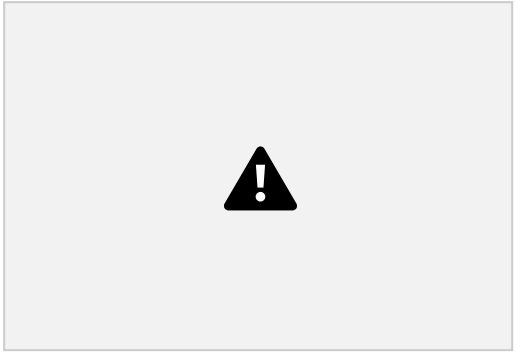


```
class Employee{  String name;
  int salary;
  String cname;
```

}

String name;
int salary;

cname



name null

class Employee{

String cname;
}

Employee a=new Employee();
Employee b=new Employee();

a null salary

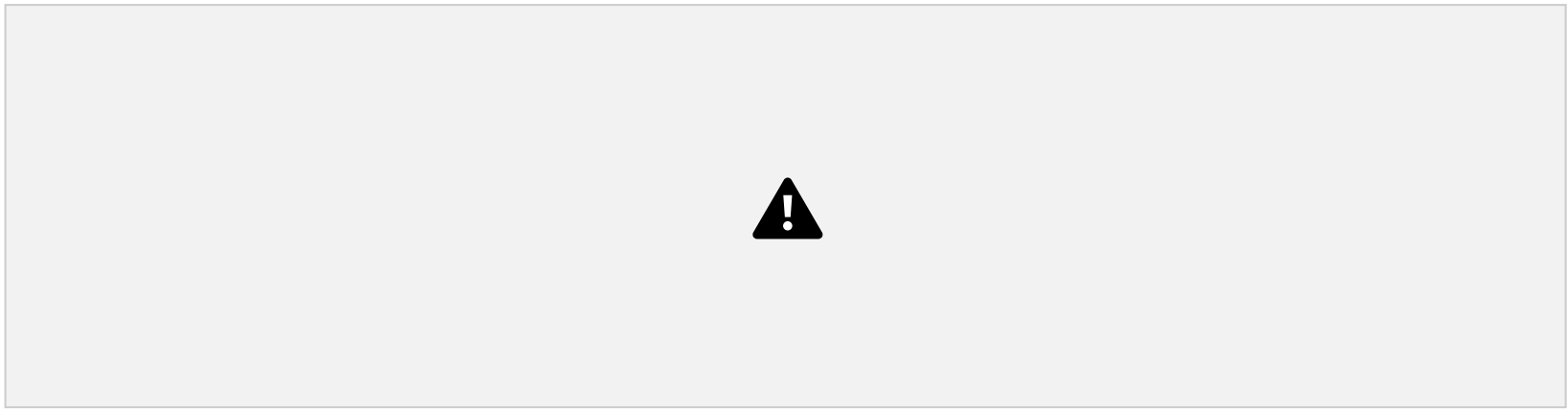
name

null cname

b nullsalary

0

static keyword can be used inside the class as: • Static variable

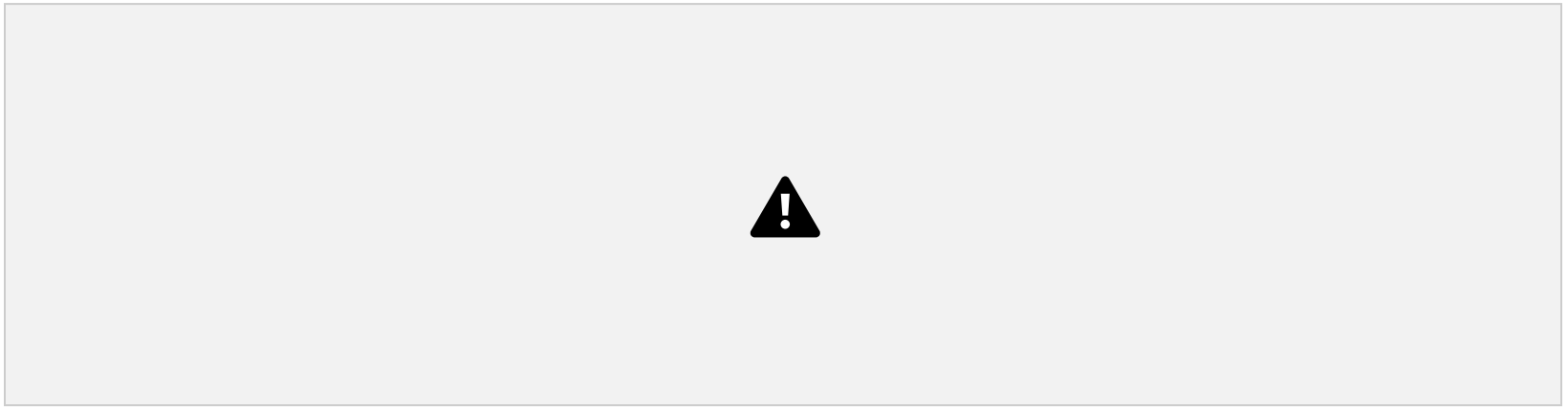


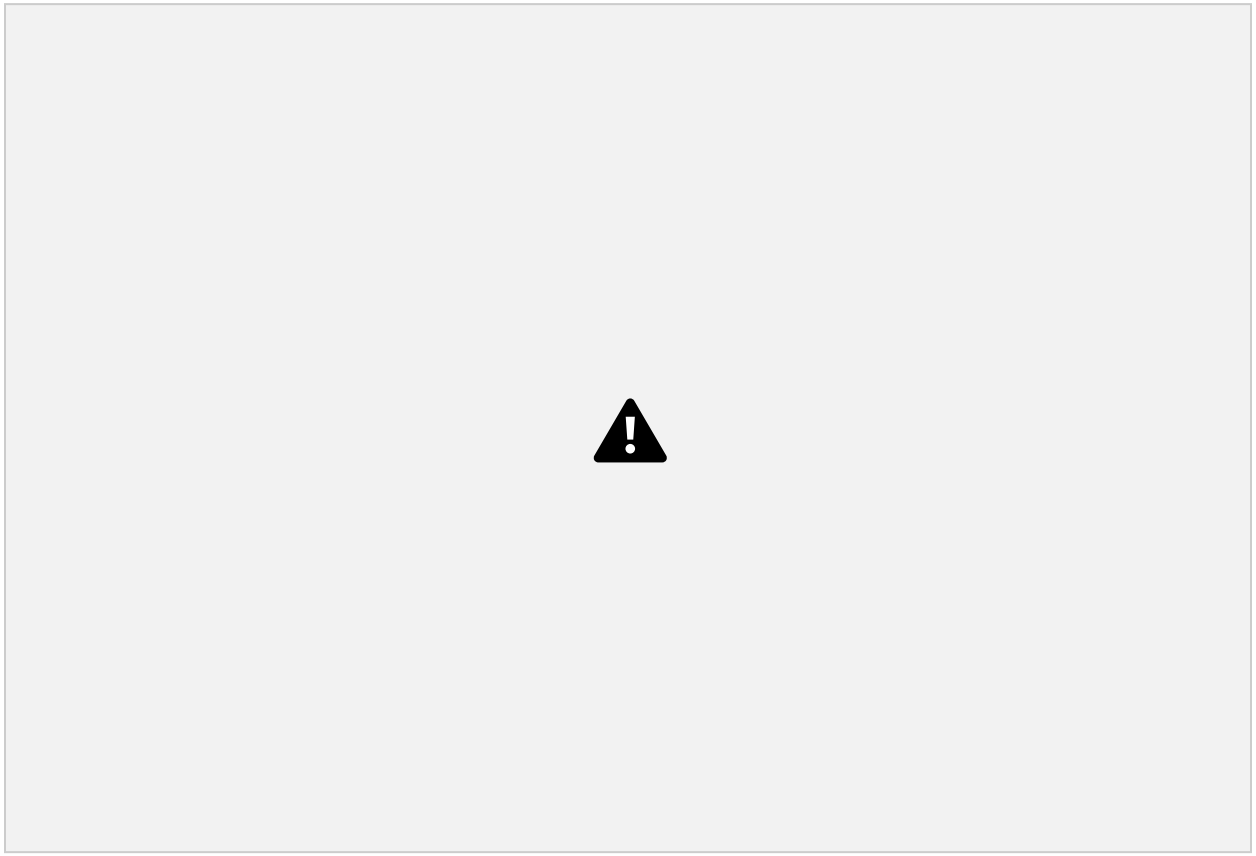
• Static method

• Static block

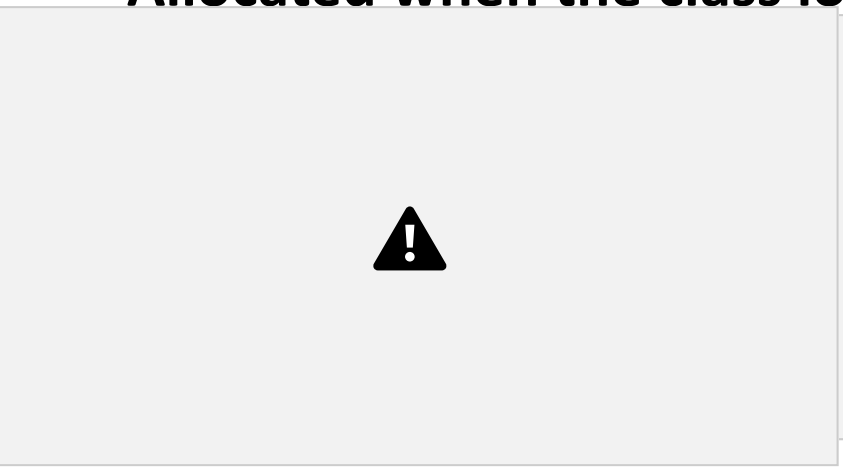
• Static nested class

A common member for every object of that class.





- **Can be accessed via Class-Name and Object reference.**
- **Allocated when the class loads in memory.**



class **Employee**{

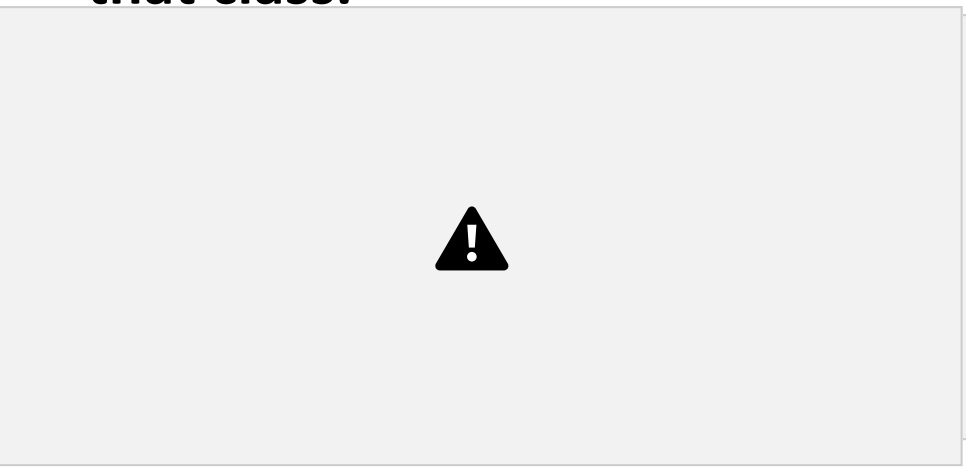
String name;

int salary;

```
        static String cname;
}
```

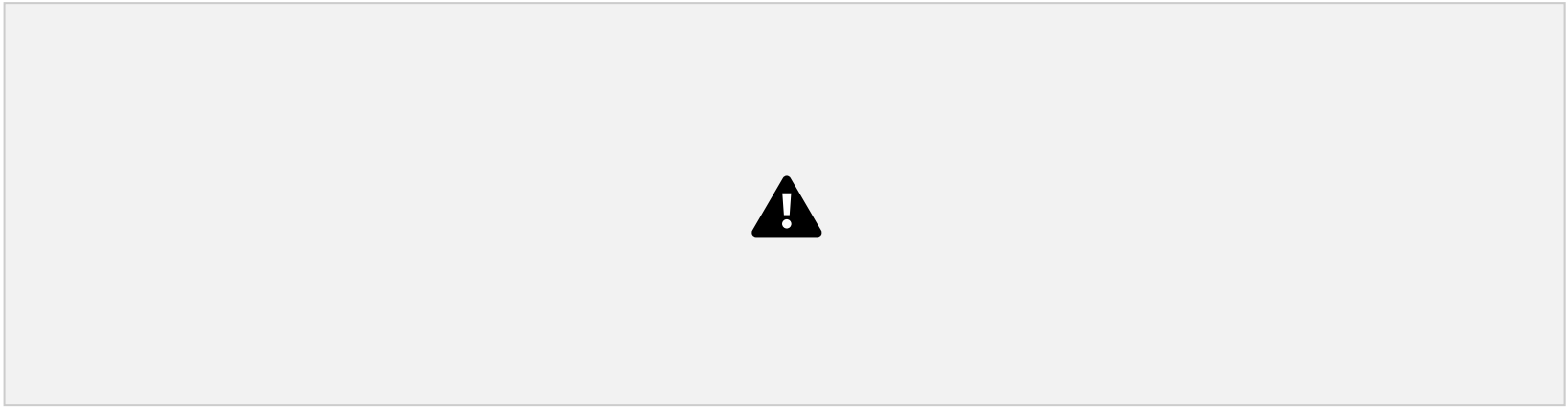
• Can be accessed via Class-Name and Object reference. • Allocated when the class loads in memory. • Can access

static member but can not access non-static member of that class.



```
class A{
static void methodName(){
//body
}
}
```

- **Automatically executes when the class loads in memory.**
- **Only executes once.**



- **Can access static member but can not access non-static member of that class.**
- **Mostly used for initializing the static variable.**

```
class A{
static {
```

```
        //body
        }
}
```

Static member

- **Also known as Class Member.**
- **Common for Every Object of the class.**

- **Can be accessed via Class name or Object reference.**
- **Allocated when class loads in memory.**
- **Static members can be accessed from static and non-static context both.**

[redacted]

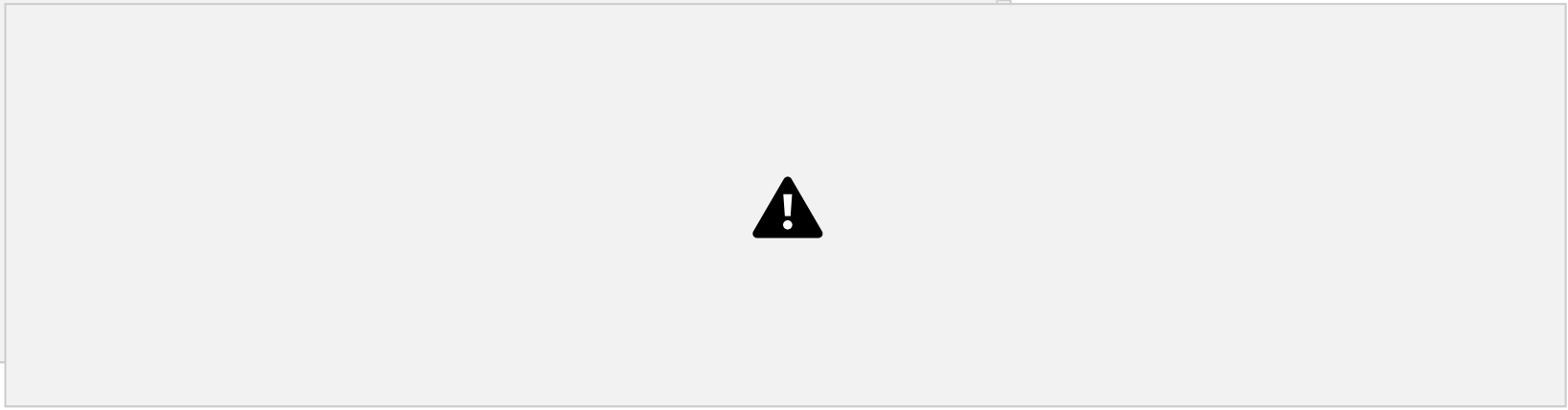
- Also known as Instance Member.

Individual for Every Object of the class.

- Can be accessed via Object reference only.
- Allocated when object created in memory.

- Non-Static members can be accessed from a non-static context only.

A special method in a class that is executed automatically when an object of the class created.



class

```
Employee{
        Employee(){
        System.out.println("Object Created");
        }
}
```

```
Employee a=new Employee();
Employee b=new Employee();
```

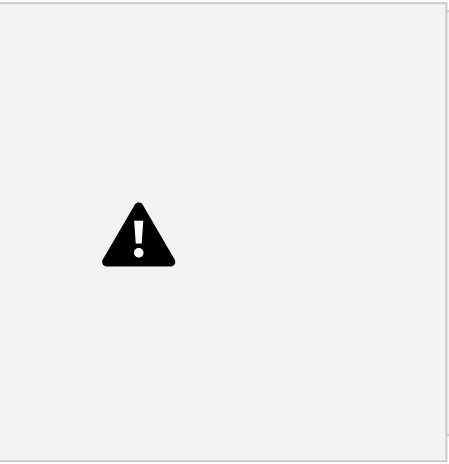
```
Employee c=new Employee();
```

- Special method in a class.
- Name must be same as class name.
- Can not declare return type (even, void not allowed).
- Invoked implicitly with object construction.
- Mostly used to initialize the object of the class.

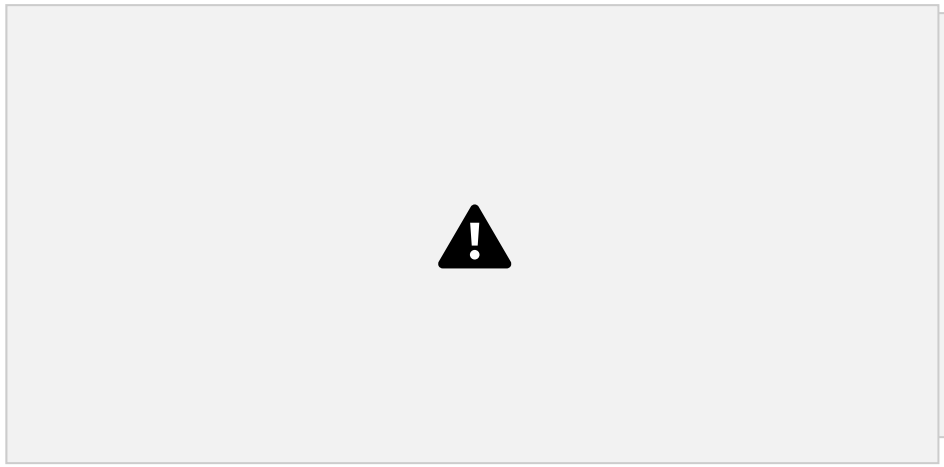
- Normal method in a class.
- Name is the choice of programmer.
- Can declare return type or void.
- Invoked explicitly by programmer's choice.
- Used to do anything that programmer needs.

Non-Parameterized

Parameterized

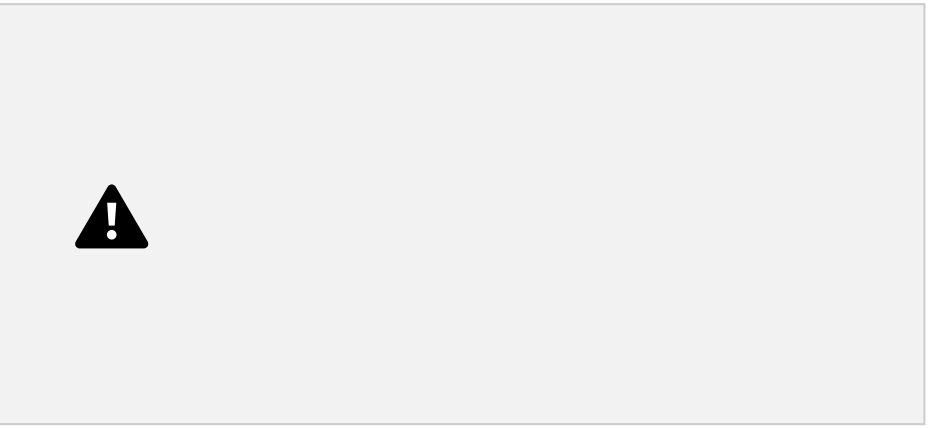


```
class
Employee{
Employee(){
 //body
}
}
```



//body
}
}

class **Employee**{



Employee(String name){

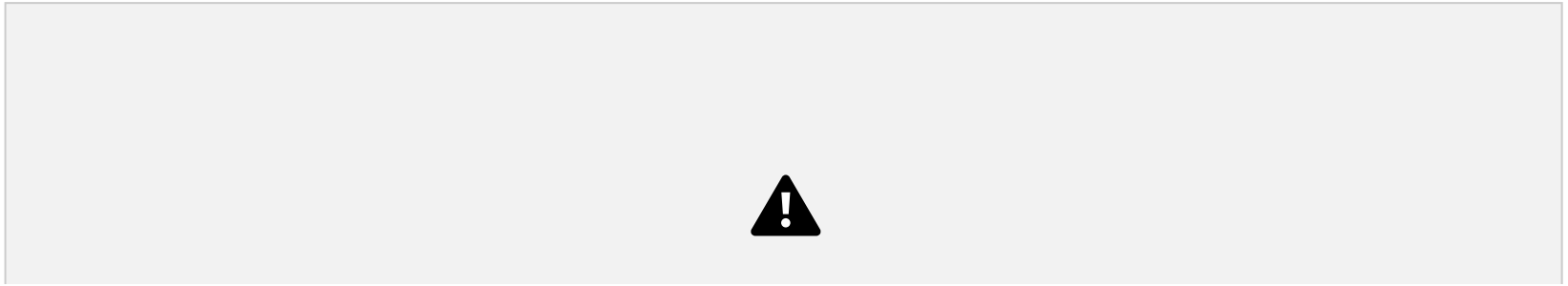
```
class Employee{
Employee(){
                    //body
```

```
        }
         Employee(String name){
         //body
        }
}
```

```
class Employee{
Employee(){
                                    //body
                                    }
```

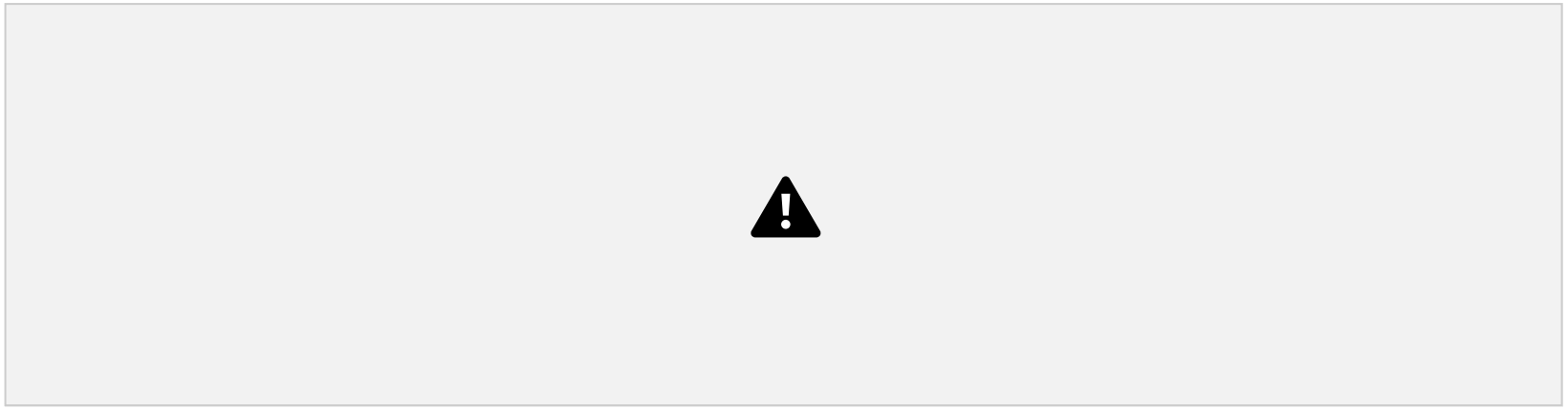
```
    Employee(String
   name){  this();
    //body
   }
}
```

• Can not chain more than one constructor in a constructor. • Constructor chaining statement must be the first statement. •



Recursive constructor chaining is not allowed. • “this” and “super” constructor chaining can not be used together in a constructor.

• **A special block in a class that is executed automatically before constructor when an object of the class created. •**

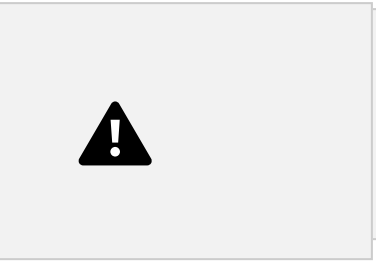


This block having no name that's why also known as anonymous block of the class.

```
class Employee{
Employee(){
 //body
}
```

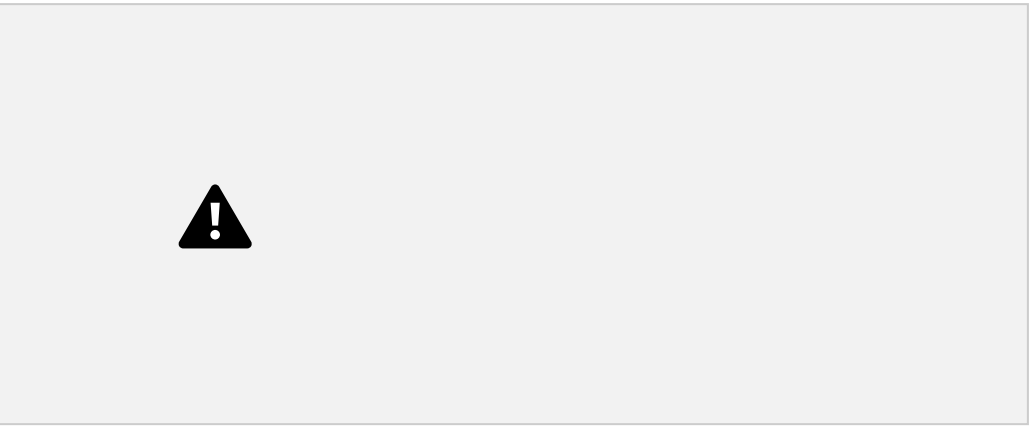
```
        {
         //body
        }
}
```

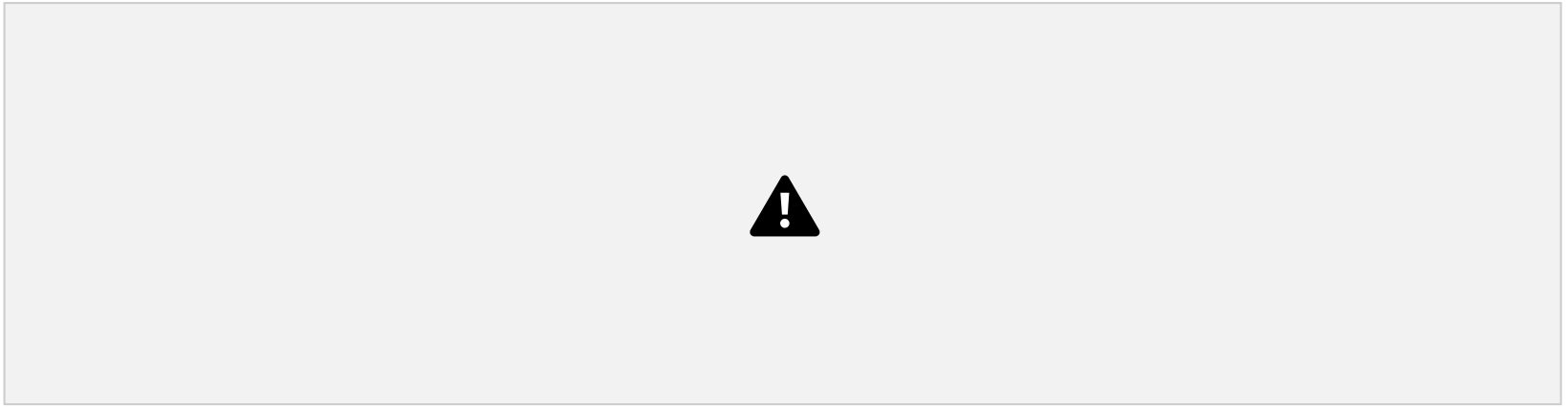
Mechanism of creating new class from old one is called Inheritance.

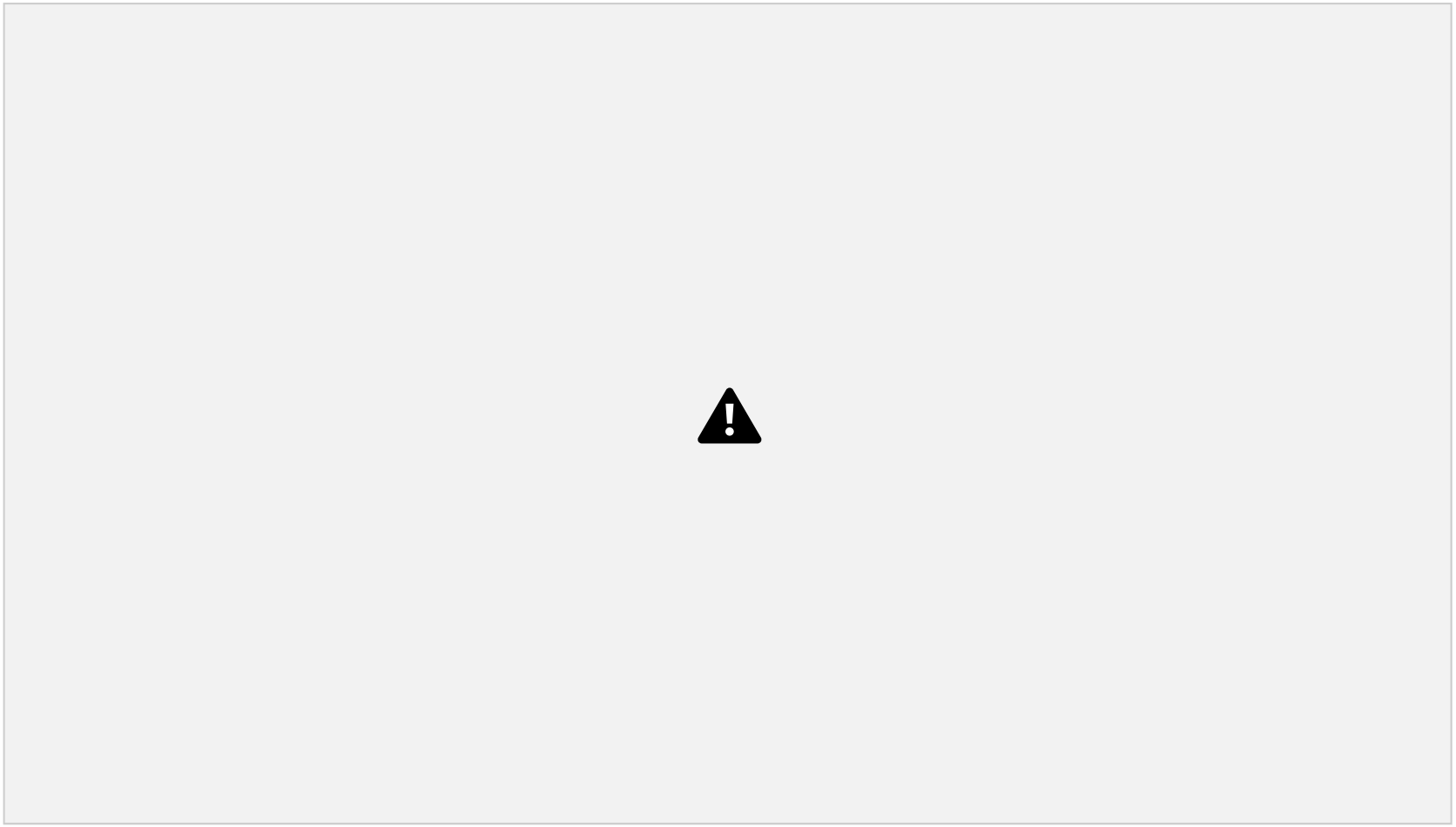


```
//members
}
```


```
class Student extends
  Human{ //members
}
```

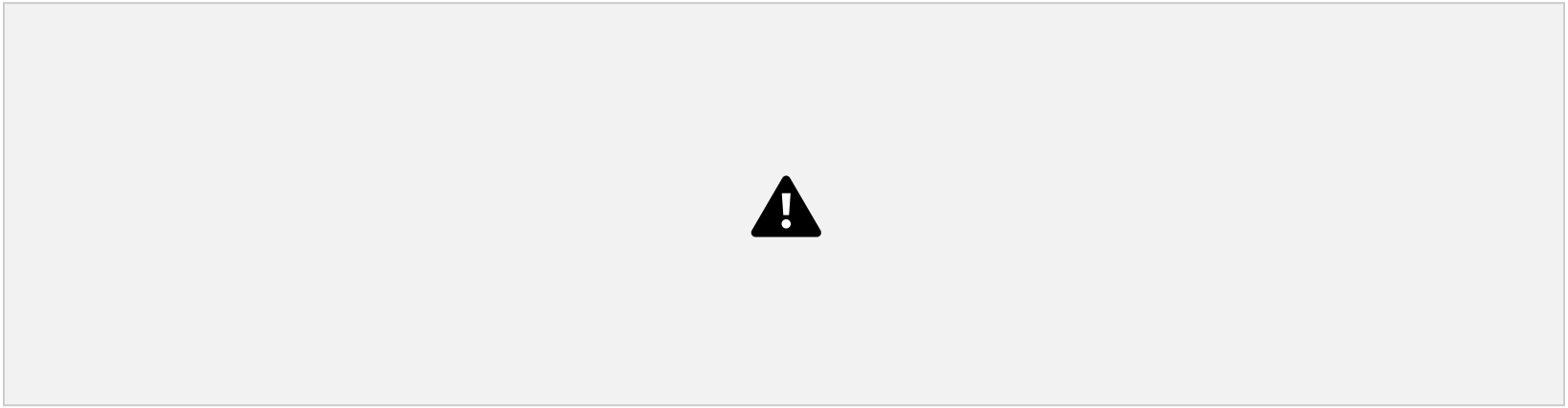






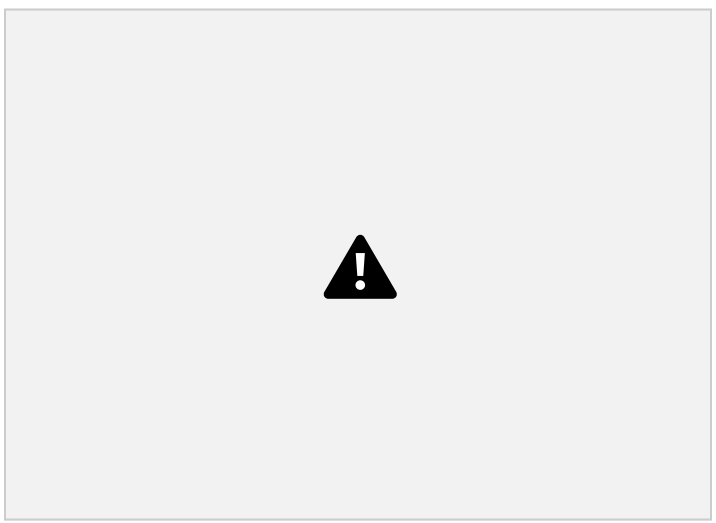
In Java classes Multiple Inheritance is NOT supported. But in Java Interfaces it is supported.

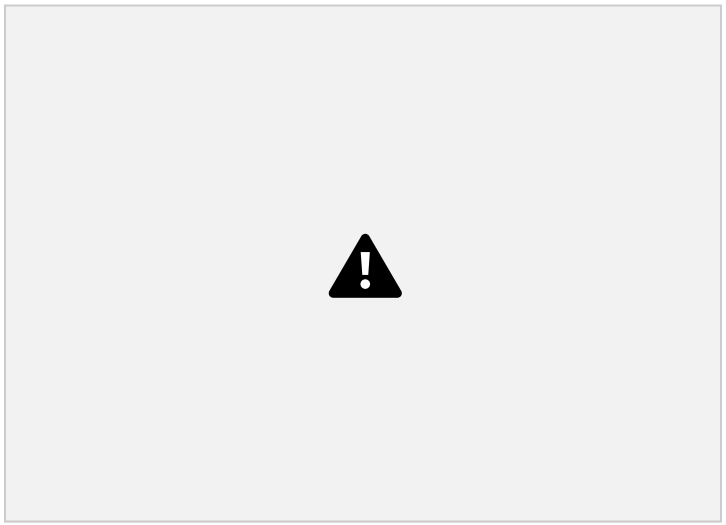
In Real World:
Many forms of doing a job.



Examples- 1) Display has many forms. 2) Fly has many forms.

In Programming:
Many methods having same name.

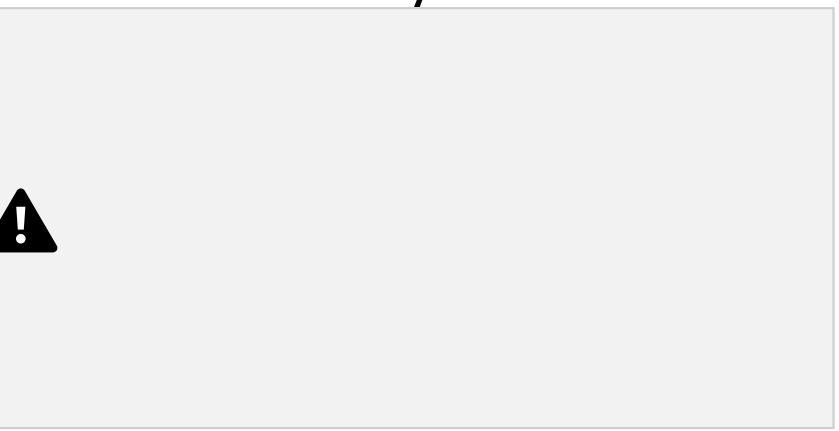




Polymorphism

Compile-Time(Static)
Polymorphism

Achieved by



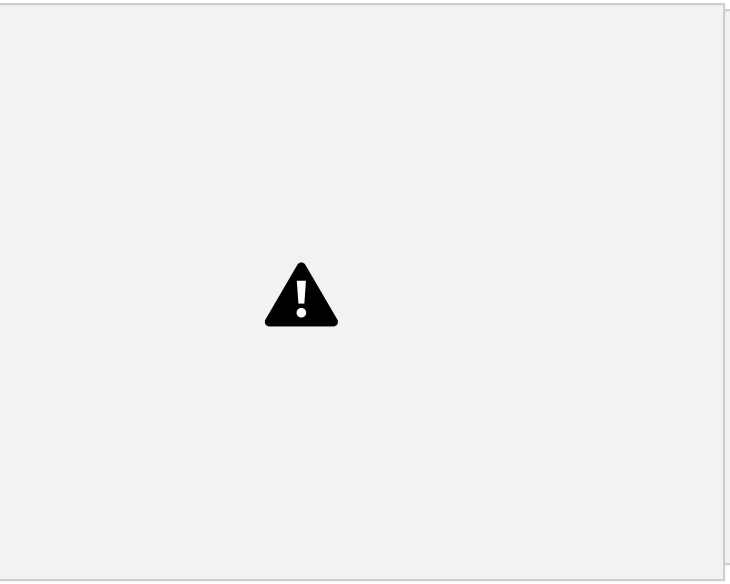
Run-Time(Dynamic)

Polymorphism

Achieved by
Method Overriding

Having more than one method with same name and different arguments in a class is known as **Compile-Time polymorphism / Static Polymorphism.**

class **Addition**{



}
}

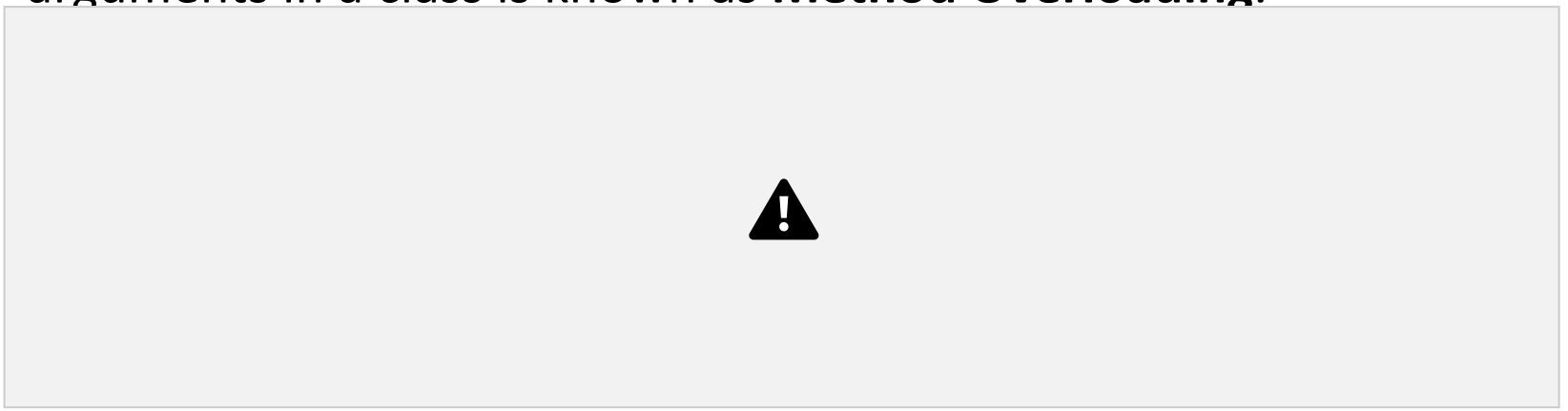
In

compile-time polymorphism, the compiler binds the method call to the method declaration at compile time, and the JVM executes it as bound.

void

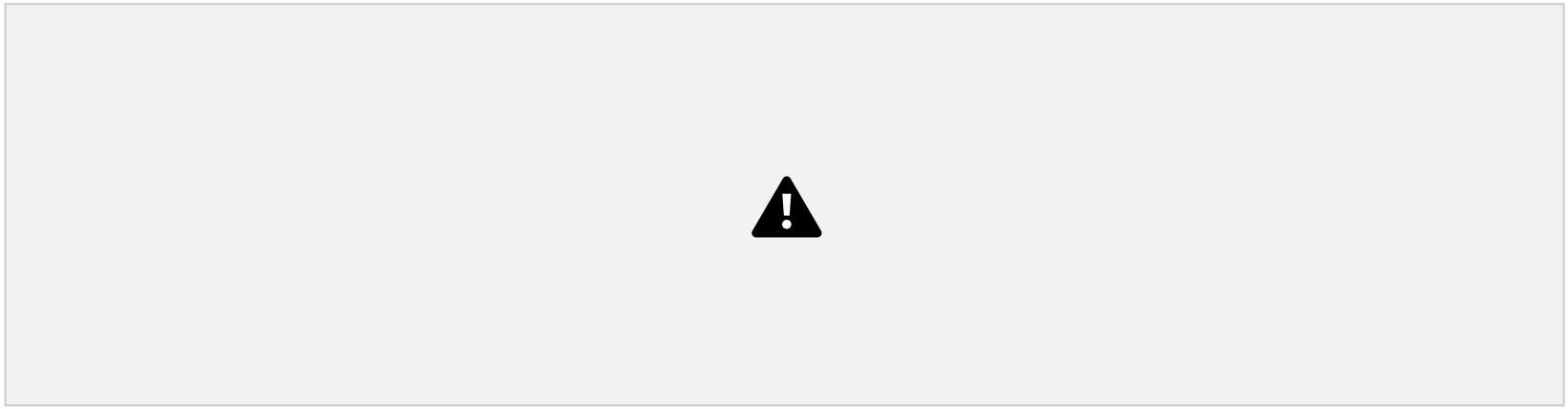
add(int x,int y){
//body
}
void add(String x,String y){ //body

Having more than one method with same name and different arguments in a class is known as **Method Overloading**.



- **Method name must be same.**
- **Method argument must be different.**
- **Method return type can be same or different.**
- **Inheritance is optional.**
- **Method can be static or non-static.**

Having same signature non-static methods in super and

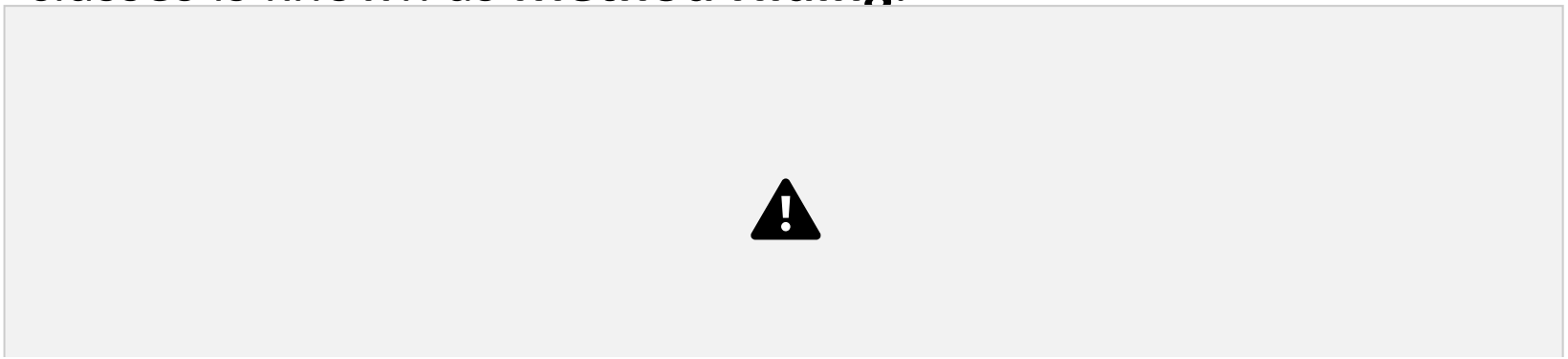


sub classes, and calling the sub class method from the super-class reference, then we achieve **Run-Time polymorphism**.

Having same signature non-static methods in super and sub classes is known as **Method Overriding**.

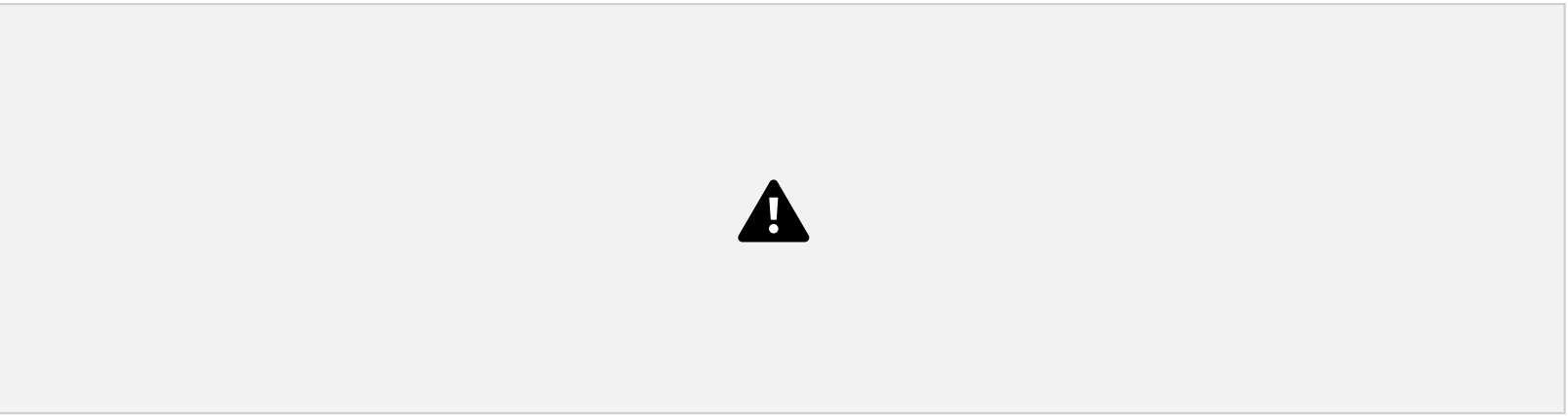
- **Method name must be same.**
- **Method argument must be same.**
- **Method return type must be same.**
- **Inheritance is compulsory.**
- **Access modifier must be same or strong.**
- **Method must be non-static.**

Having same signature static method in super and sub classes is known as **Method Hiding**.



- Method name must be same.
- Method argument must be same.
- Method return type must be same.
- Inheritance is compulsory.
- Access modifier must be same or strong.
- Method must be static.

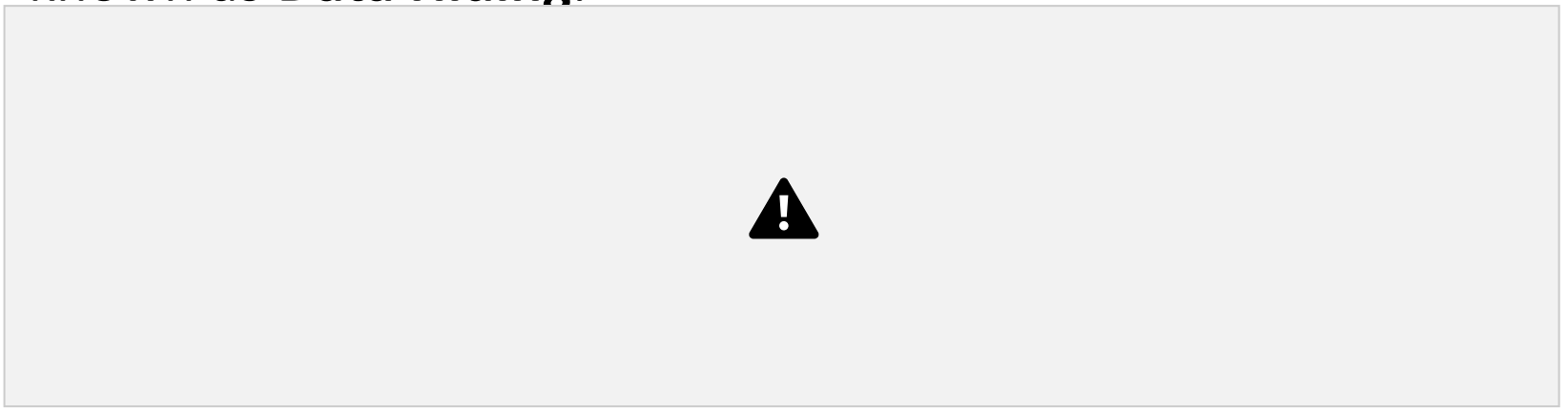
Sub-class with the Same Signature as in Super-class



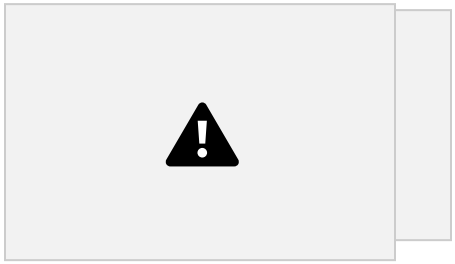
Overrides error

error Hides

Having same name variable in super and sub classes is known as **Data Hiding**.



- **Variable name must be same.**
- **Variable type can be same or different.**
- **Inheritance is compulsory.**
- **Access modifier can be same or different.**
- **Variable can be static or non-static.**



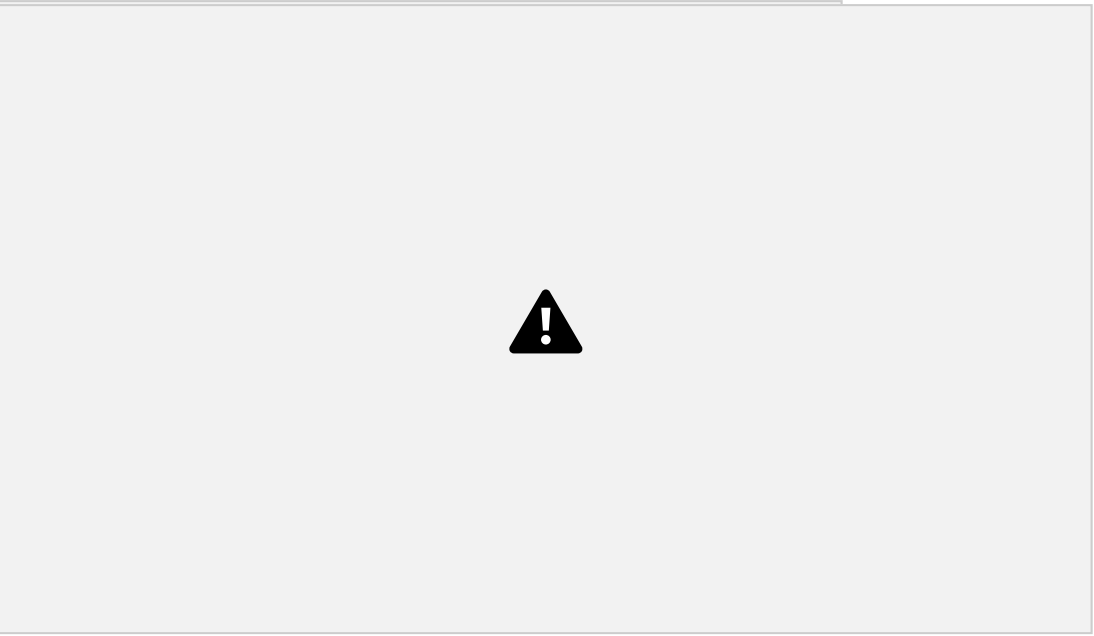
class **A**{ void m1(){ } }

B x=new B();

A y=x; [upcasting]

class **B** extends A{ void m2(){ }

}



~~B z=y~~ ; [compile error]

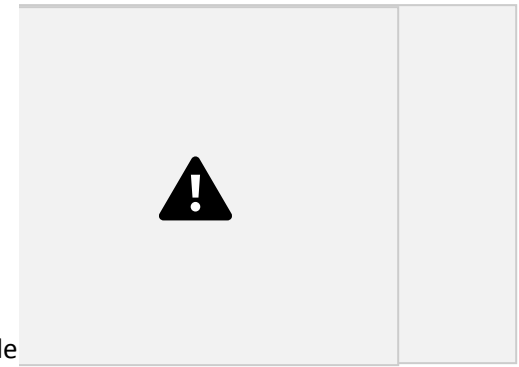
B z=(B)y; [downcasting]

A y=new B() ; [upcasting]

~~B x=new A()~~ ; [compile error] B

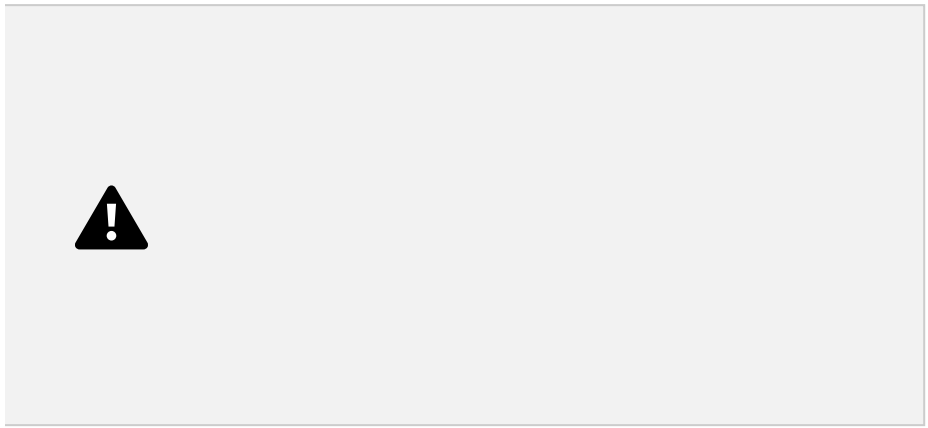
x=(B)new A() ; [ClassCastException]

private double a;
public void **area**(){
a=3.14 X 5 X 5 ;
}
public void **show**(){
SOP("Circle's Area: "+a) ; }
}



public class **Rectangle**{
private double v;
public void **m1** (){
v=10 X 7 ;
}
public void **m2**(){
SOP("Rectangle's Area: "+v) ;
}

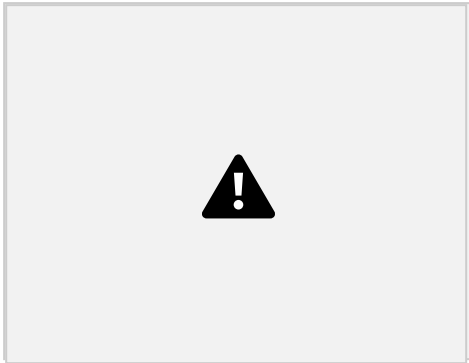
}



```
public class Triangle{
 private double ar;
 public void calculate(){
 a= 15 X 8 / 2;
 }
 public void display(){
 SOP("Triangle's Area: "+ar) ;
}
}
```

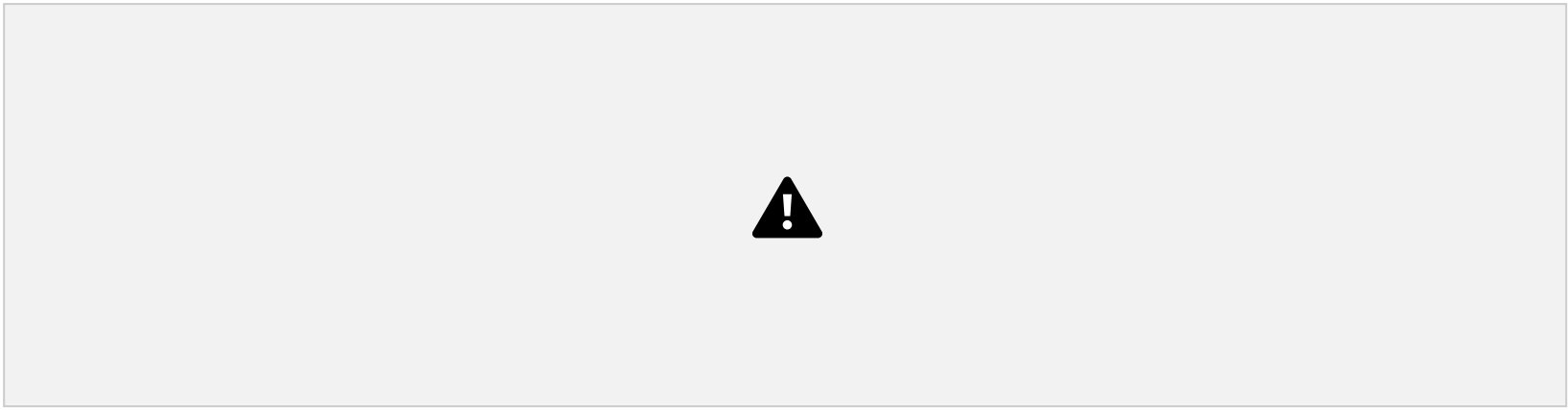
```
            public class ShapeApp {
             public static void main(String [] s){
            Circle c=new Circle();
             c.area();
             c.show();
```

```
 Rectangle r=new Rectangle();
r.m1();
 r.m2();
 Triangle t=new Triangle();
t.calculate();
 t.display();
 }
 }
```



```
public class Circle extends Shape{
 private double a;
 public void findArea(){
 a=3.14 X 5 X 5 ;
 }
 public void printArea(){
 SOP("Circle's Area: "+a) ;
```

```
public class Rectangle extends Shape{
 private double v;
 public void findArea(){
 v=10 X 7 ;
 }
 public void printArea(){
```

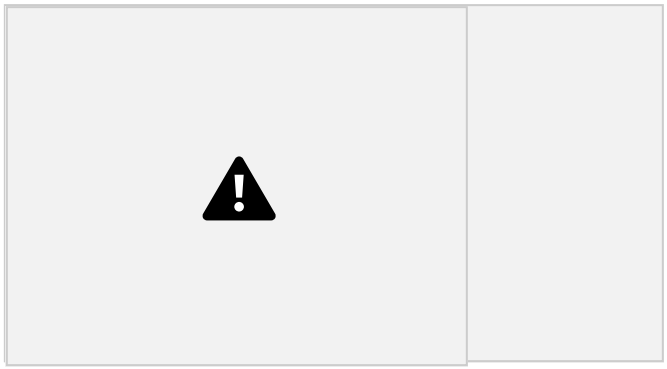


```
 }
}
```

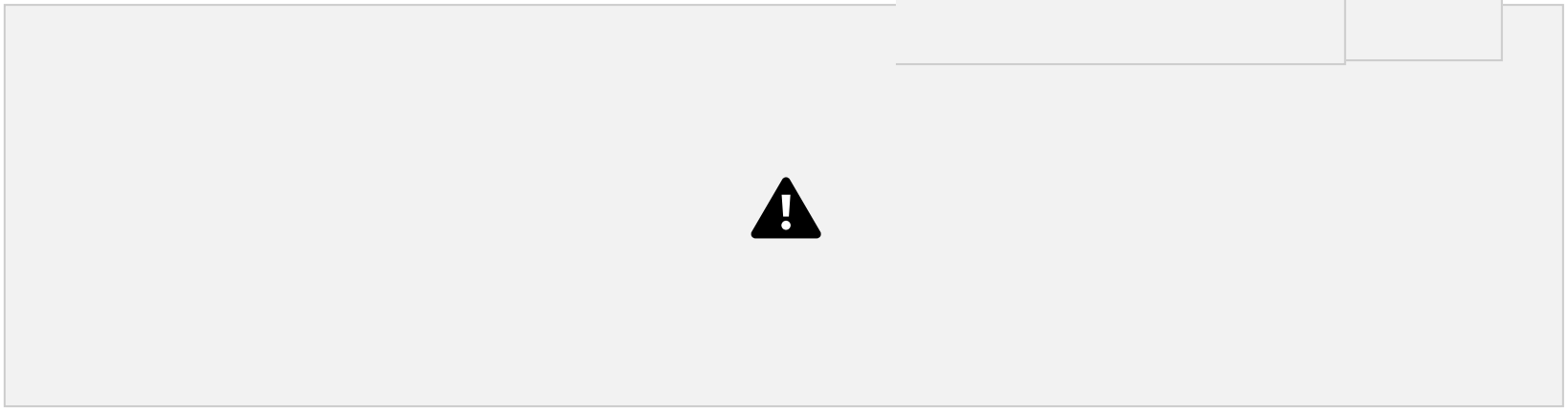
```
 SOP("Rectangle's Area: "+v) ;
}
}
```

```
public class Triangle extends Shape{
 private double ar;
 public void findArea(){
 a= 15 X 8 / 2;
 }
 public void printArea(){
 SOP("Triangle's Area: "+ar) ;
}
}
```

```
public class ShapeApp {
 public static void main(String [] s){
Shape s;
 s=new Circle();
 s. findArea();
 s. printArea();
      s=new Rectangle();
 s. findArea();
 s. printArea();
   s=new Triangle();
 s. findArea();
 s. printArea();
 }
}
```



public class **Circle** extends Shape{
private double a;
public void **findArea**(){
a=3.14 X 5 X 5 ;
}
public void **printArea**(){
SOP("Circle's Area: "+a) ;
}
}



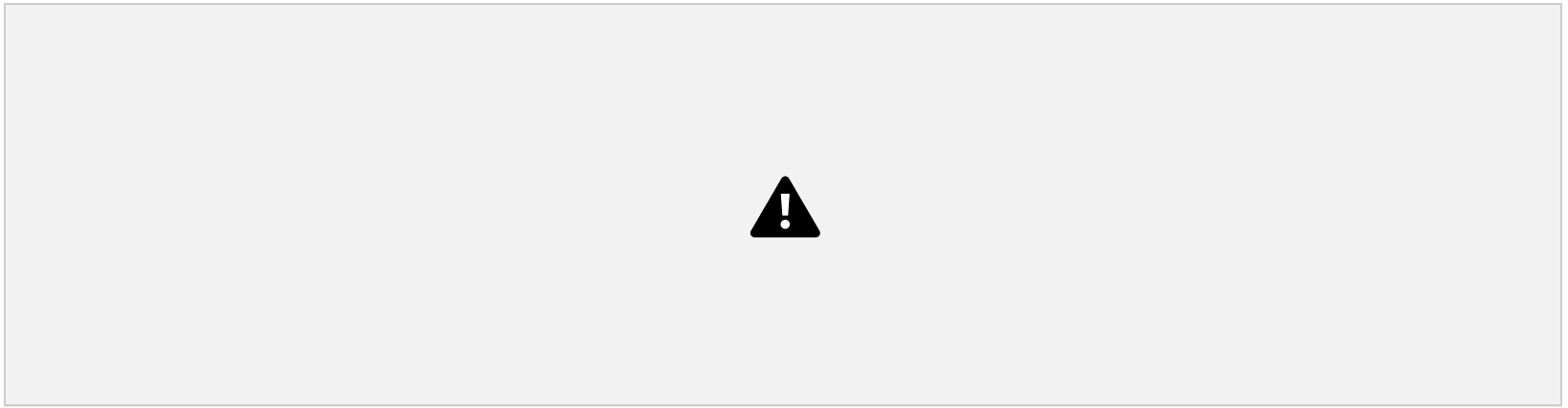
public class **Rectangle** extends Shape{ private double v;
public void **findArea**(){
v=10 X 7 ;
}
public void **printArea**(){
SOP("Rectangle's Area: "+v) ;
}
}

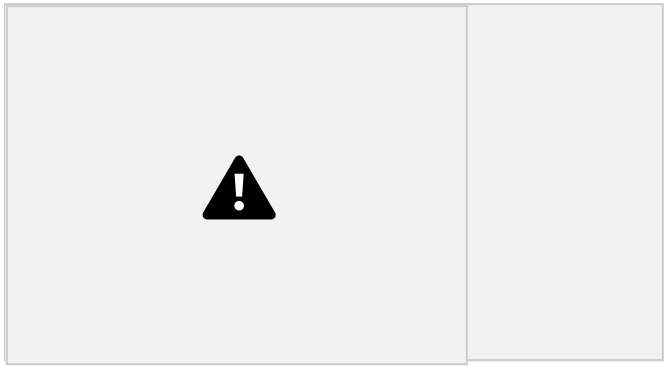
public class **Triangle** extends Shape{
private double ar;
public void **findArea**(){
a= 15 X 8 / 2;
}
public void **printArea**(){
SOP("Triangle's Area: "+ar) ;
}
}

abstract

```
public class ShapeApp {
 public static void main(String [] s){  Shape s= new Shape() ;
                                         s=new Circle();
                                          s. findArea();
                                          s. printArea();
                                          s=new Rectangle();
                                          s. findArea();
                                          s. printArea();
                                          s=new Triangle();
                                          s. findArea();
                                          s. printArea();
                                          }
                                         }
```

- **A class that can not be instantiated.**
- **A class that is used to access its sub-classes only. • A class that is used to achieve Run-time polymorphism.**

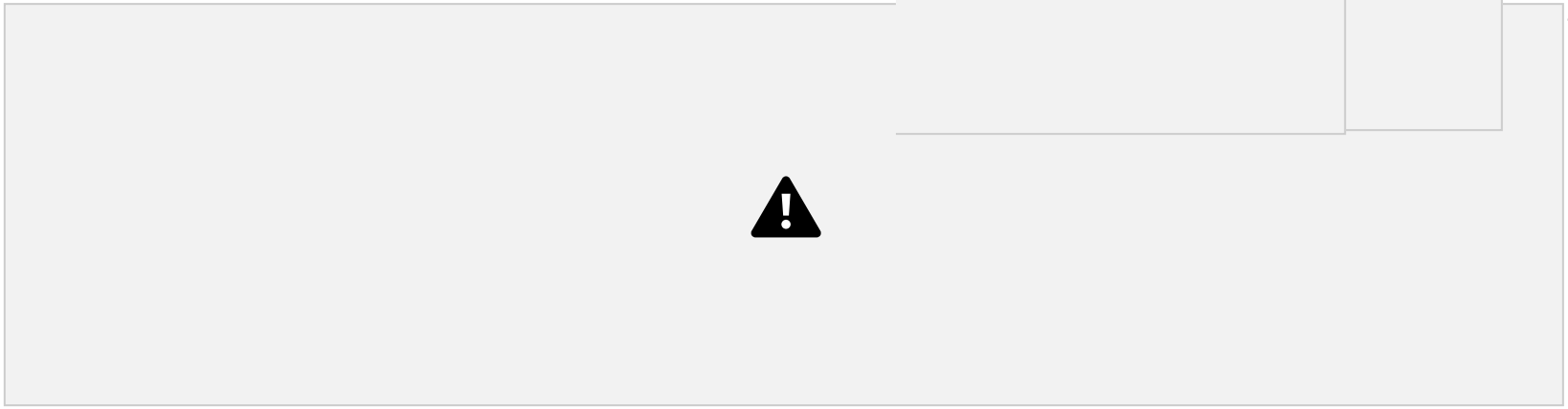




public class **Circle** extends Shape{
private double a;
public void **findArea**(){
a=3.14 X 5 X 5 ;
}
public void **printArea**(){
SOP("Circle's Area: "+a) ;
}
}

Rectangle extends Shape{
private double v;
public void **findArea**(){
v=10 X 7 ;
}
public void **printArea**(){
SOP("Rectangle's Area: "+v) ;
}
}

public class **Triangle** extends Shape{
private double ar;
public void **findArea**(){
a= 15 X 8 / 2;
}
public void **printArea**(){
SOP("Triangle's Area: "+ar) ;
}
}



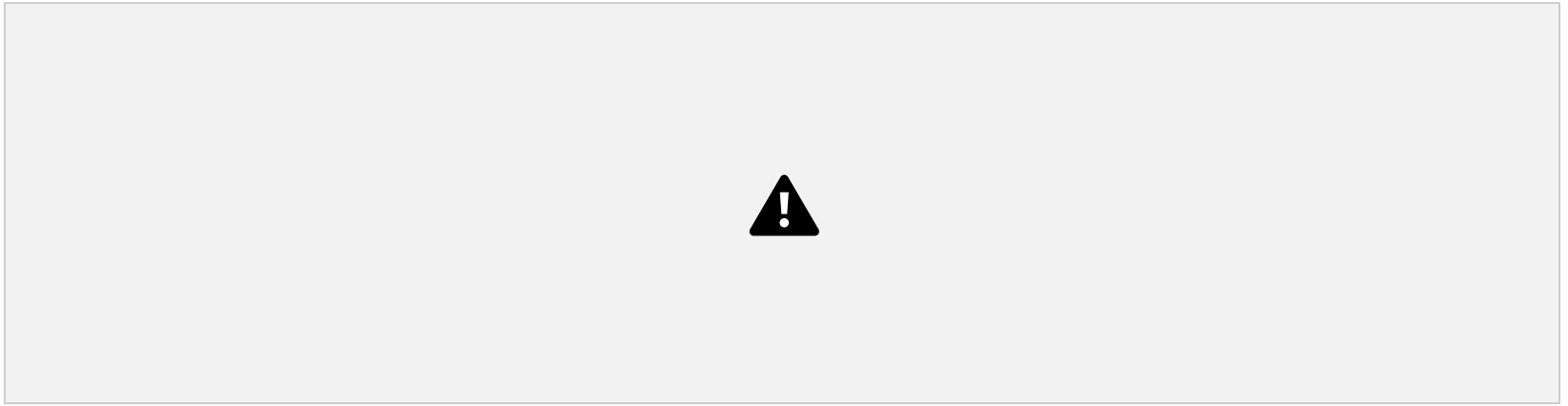
public class

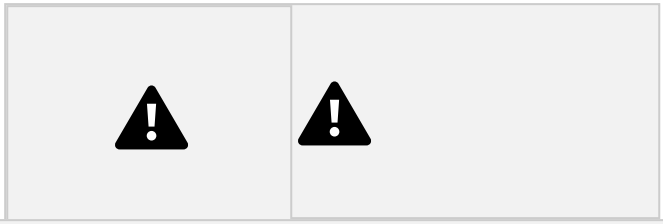
abstract abstract
abstract

```
s=new Circle();
 s. findArea();
 s. printArea();
 s=new Rectangle();
 s. findArea();
 s. printArea();
 s=new Triangle();
 s. findArea();
 s. printArea();
 }
}
```

```
public class ShapeApp {
 public static void main(String [] s){  Shape s= new Shape() ;
```

- **A method without body.**
- **A method that must be overridden in its non-abstract(Concrete) sub-class.**





public class **A** {
public void **show**(){
SOP("Hello A") ;

implements Shape{
private double ar;
public void **findArea**(){
a= 15 X 8 / 2;
}
public void **printArea**(){
SOP("Triangle's Area: "+ar) ;
}
}

public class
Rectangle extends A implements Shape{ private double v;
public void **findArea**(){
v=10 X 7 ;
}
public void **printArea**(){
SOP("Rectangle's Area: "+v) ;
show();
}
}

abstract **interface** **abstract**
abstract

public class
Triangle

```
s. findArea();
s. printArea();
}
}
```

```
public class Test{
 PSVM(String [] s){
Shape s= new Shape() ;
s=new Circle();
s. findArea();
s. printArea();
s=new Rectangle();
s. findArea();
s. printArea();
s=new Triangle();
```

Interface

Abstract class

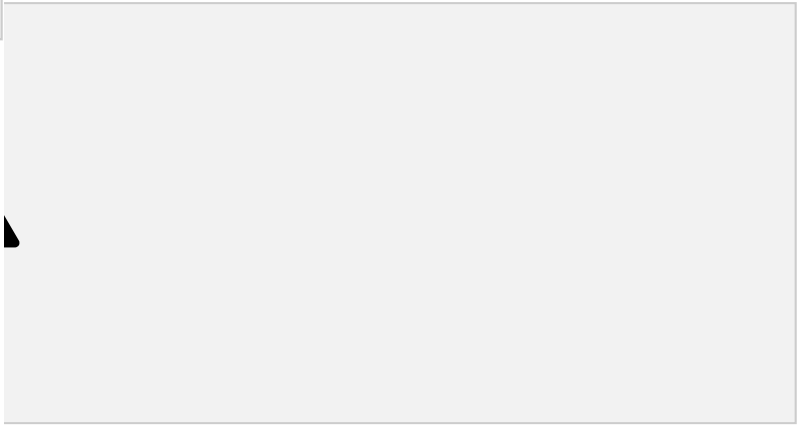
• It is 100% abstract class by default. • Supports multiple inheritance. • Does not support constructor. • Method in interface is by default abstract and public.

• Variable in interface is by default public, static and final.

• To inherit an interface in class, we use **implements** keyword and To inherit an interface in interface, we use **extends** keyword.

• From Java-8, **default method** and **static method** are allowed in the interface. • From Java-9, **private** method is also allowed in the interface.

• It can be 100% abstract or not. • Does not



supports multiple inheritance. • Supports constructor.

• Method in abstract class is not by default abstract and public.

• Variable in abstract class is not by default public, static and final.

- To inherit an abstract class, we use **extends** keyword.

- **Final class** (A class that can not be Inherited)
- **Final method** (A method that can not be overridden or hidden)
- **Final variable** (A variable, who's value can not be changed)

final class

```
final class A{
        //members
}
```

final method

```
class A{
        final void method(){
        //body
        }
}
```

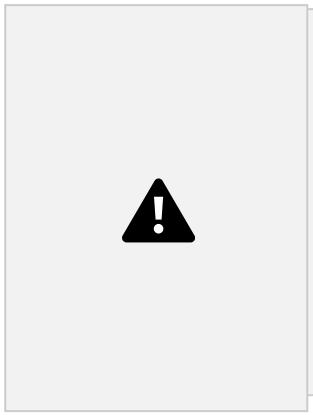
final variable

```
class A{
        final int x=10;
        void method(){
                final int z=20;
        }
}
```

class Test{

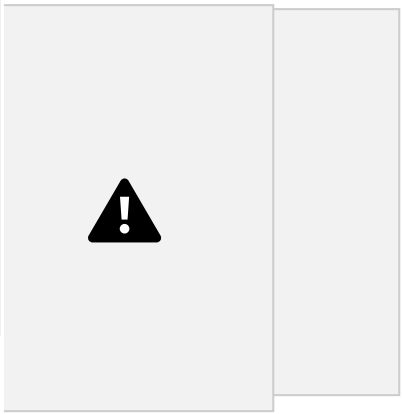
class Test{

class Test{



final

```
int a;  Test(){
 a=10;
 }
```



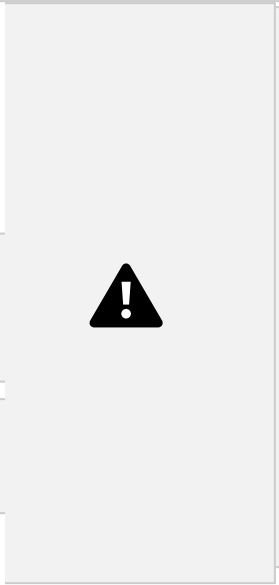
final

```
int a;  {
 a=10;
```

```
}
```

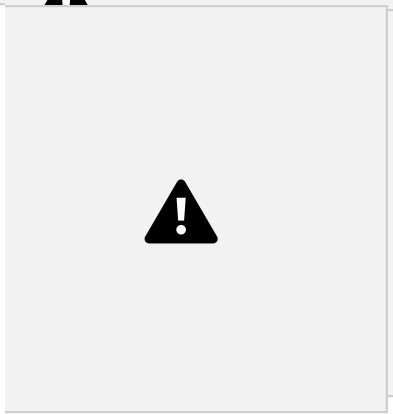
```
static {
 a=10;
 }
}
```

static final int a



class

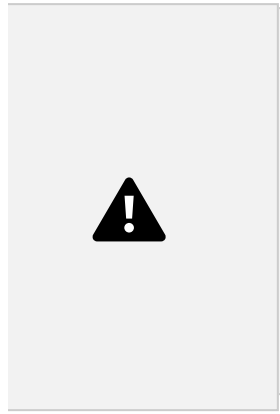
```
Test{  final int a;
Test(){
 a=10;
 }
 {
 a=10;
 }
}
```



```
class Test{
 static final int a;
Test(){
 a=10;
```

```
 }
}
```

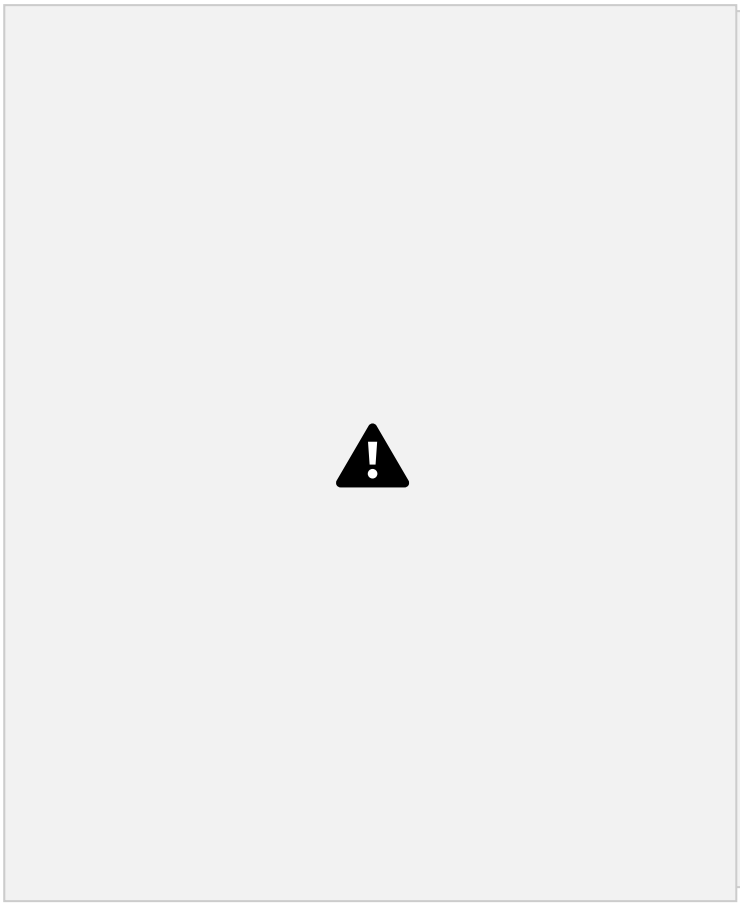
```
static{
 a=10;
 }
}
```



- **Non-static nested class**
- **Static nested class •**

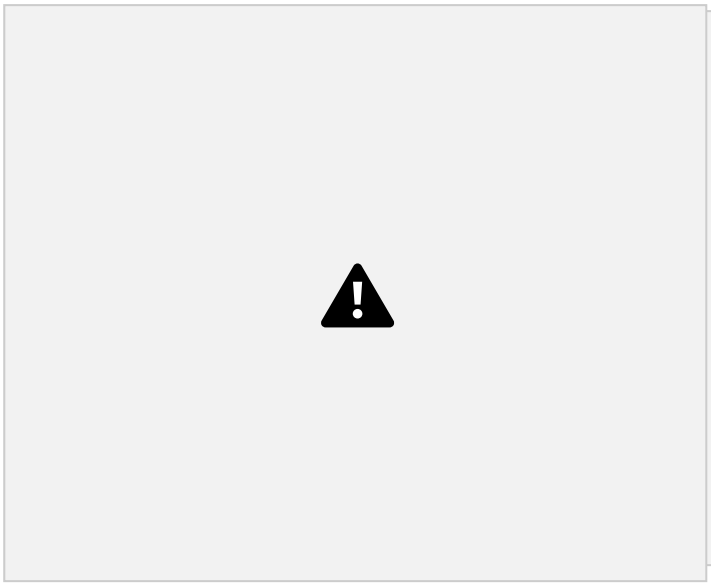
Local nested class

- **Anonymous nested class**

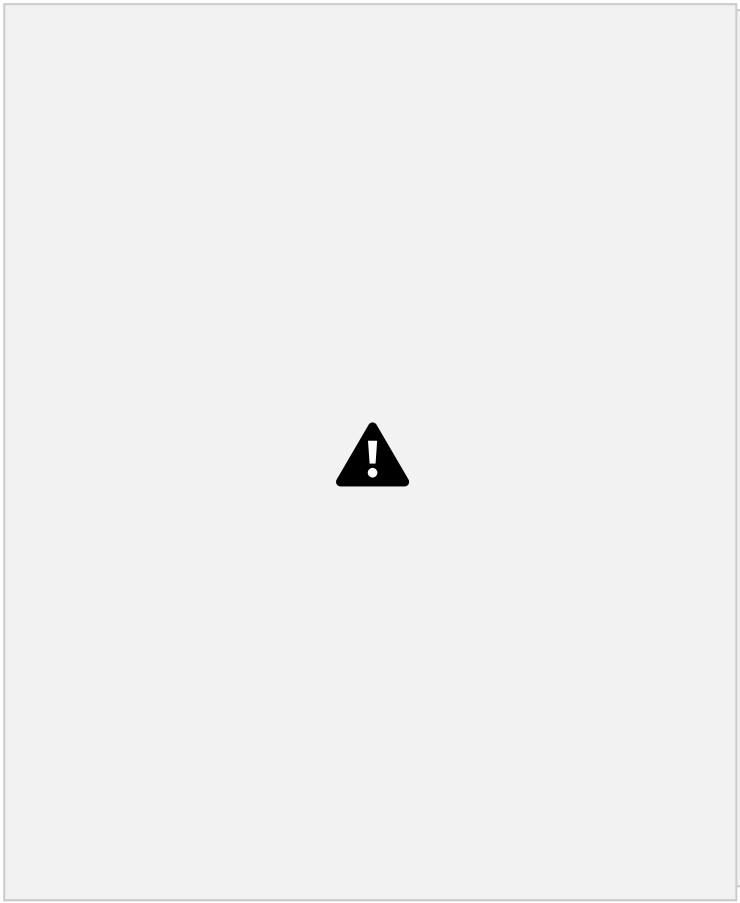


```
void m1(){
 B b=new B();
```

```
class A{
 SOP(b.z);
 b.m();
}
class B{
 int z;
 void m(){
SOP("hi B");
 }
}
}
```



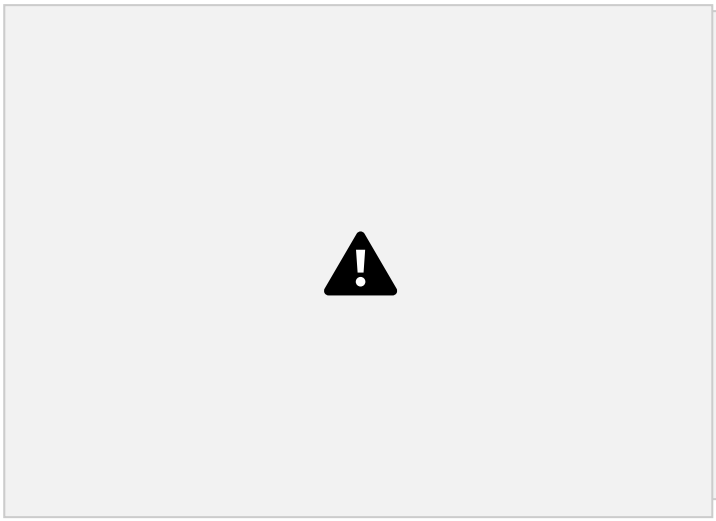
```
class Demo{
        PSVM(-){
         A a=new A();
         A.B b=a.new B();
         SOP(b.z);
         b.m();
        }
        }
```



```
class A{
}
```

```
SOP(b.z);
b.m();
}
static class B{
int z;
void m(){
        SOP("hi B");
}
}
```

```
void m1(){
 B b=new B();
```



```
class
Demo{
                                    PSVM(-){
                                        A.B b=new A.B();
                                      SOP(b.z);
                                     b.m();
                                    }
                          }
```

```
class A{
void m1(){
```

```
class B{
 int z;
 void m(){
SOP("hi B");
 }
 }
 B b=new B();
 SOP(b.z);
 b.m();
}
void m2(){
 // B b=new B(); //error
}
```

}

class Demo{

PSVM(-){
A a=new A();
a.m1();
}
}

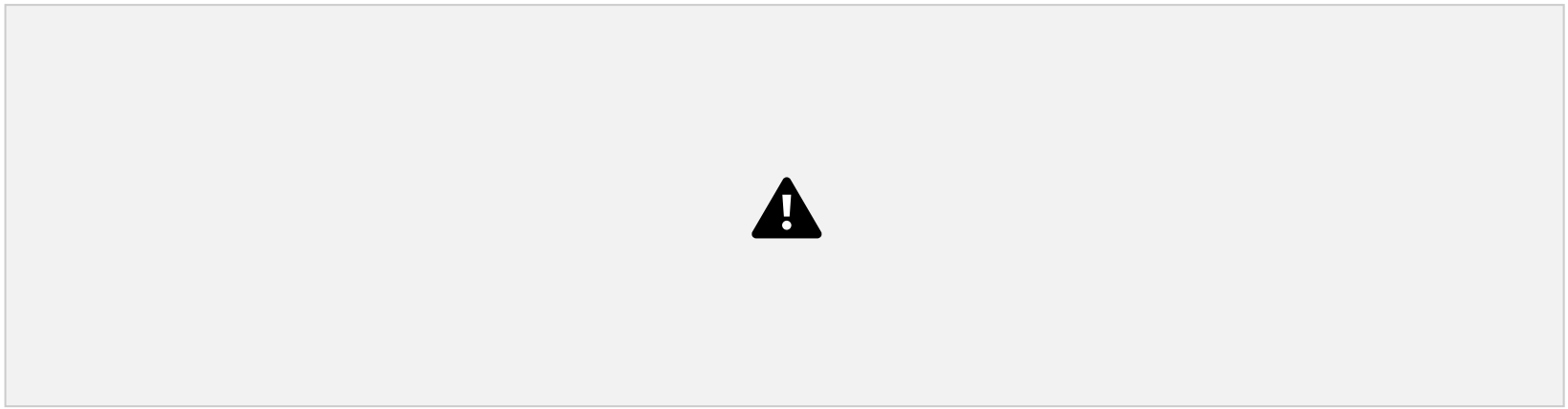
```
class A{
void m1(){
 SOP( “Hi A”);
                                    }
                                    }

                                    class Demo{
                                    PSVM(-){
                                     A a=new A() {
                                    void m1(){
                                     SOP( “Hi
```

```
INCAPP”);
}
 } ;
 a.m1();
}
}
```

Introduced in Java 8

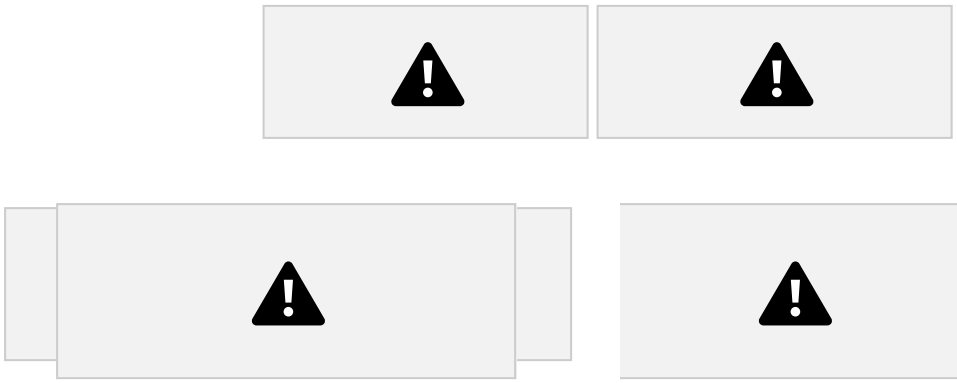
Lambda expression provides implementation of functional interface.

Functional Interface



An interface which has only one abstract method is called functional interface.

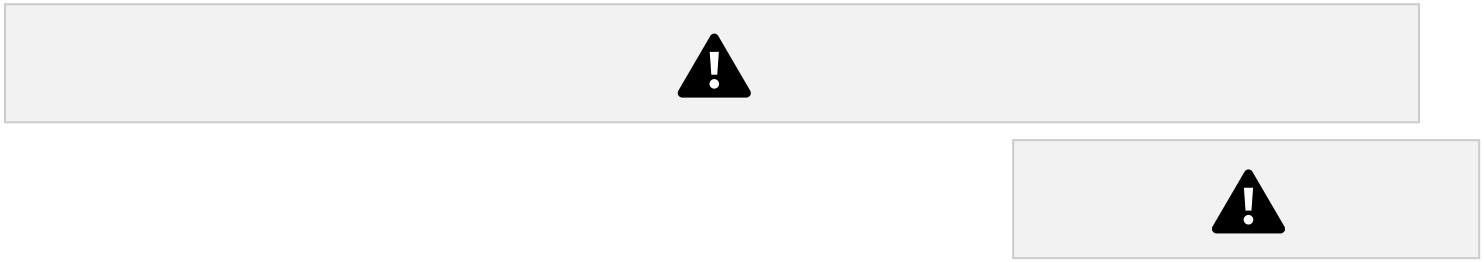
Java Lambda Expression Syntax: *(argument-list)* **->** *{body}*



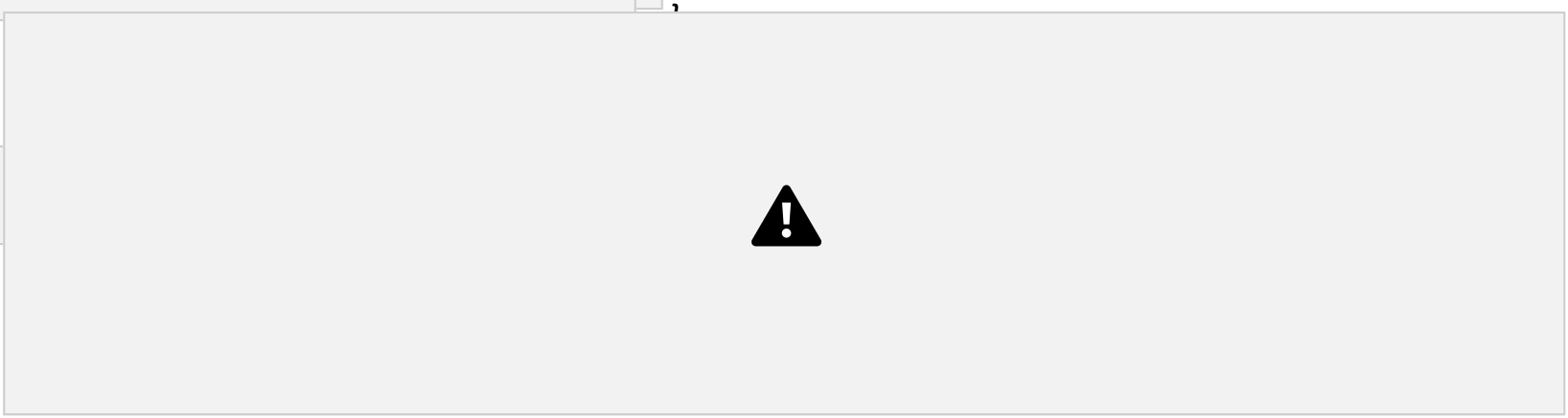
Interface that has **no** abstract method

Interface that has **only one** abstract method

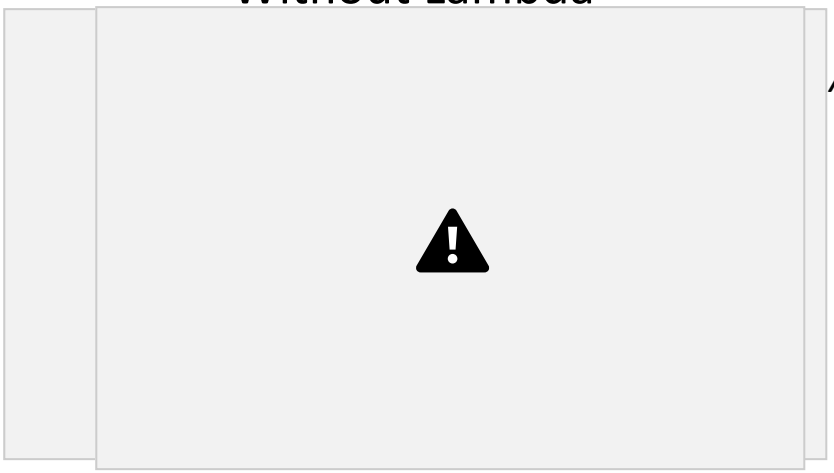
Lambda expression works with **Functional Interface** only.



```
@FunctionalInterface //It is optional
interface Addition{
public void add();
}
```



Without Lambda



```
using anonymous class
 Addition a=new Addition(){
//     public void add(){
          int a=10,b=20,r=a+b;
          System.out.println("Sum : "+r);
   }
   };
   a.add();
```

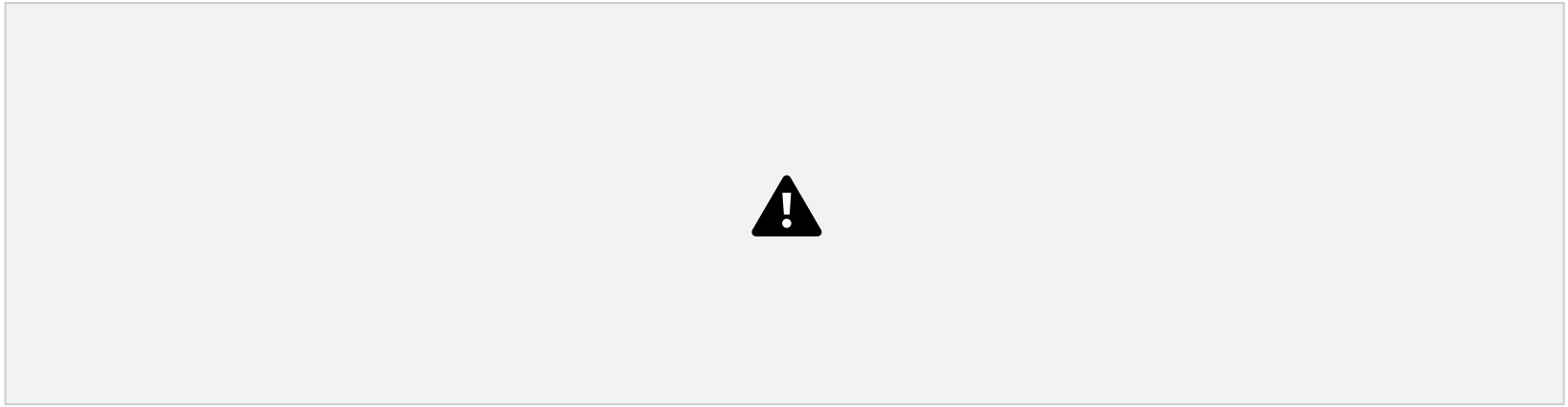
```
        int a=10,b=20,r=a+b;
        System.out.println("Sum : "+r);
};
a.add();
```

Addition a= ***( ) -> {***

Note:

If lambda expression body contains only one statement then ***curly braces are***

optional. *Addition a=()->System.out.println(“Sum : ”+(10+20));*



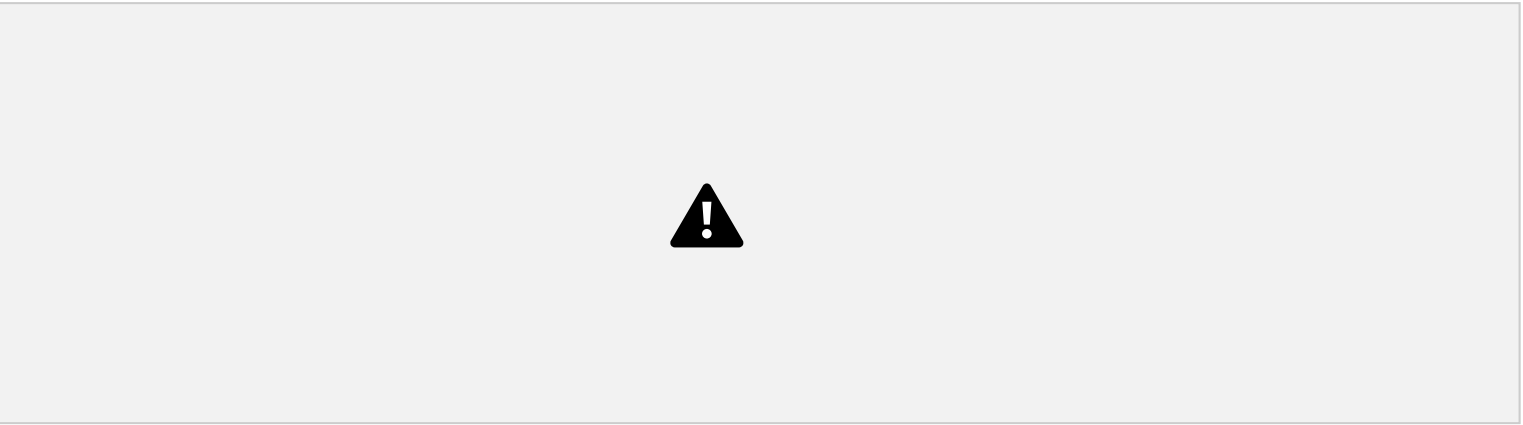
```
@FunctionalInterface
interface Addition{
    public void add(int x, int y);
}
public class LambdaExp {
    public static void main(String[] args) {
        Addition a=(x, y)->{
            int r=x+y;
            System.out.println(“Sum : ”+r);
        };
        a.add(10,20);
    }
}
```

Note: Data-type in argument is optional.

Note:

If lambda argument-list contains only one argument then round braces is optional.

```
@FunctionalInterface
interface Square{
```



```
public void findSquare(int x);
}
public class LambdaExp {
public static void main(String[]
```

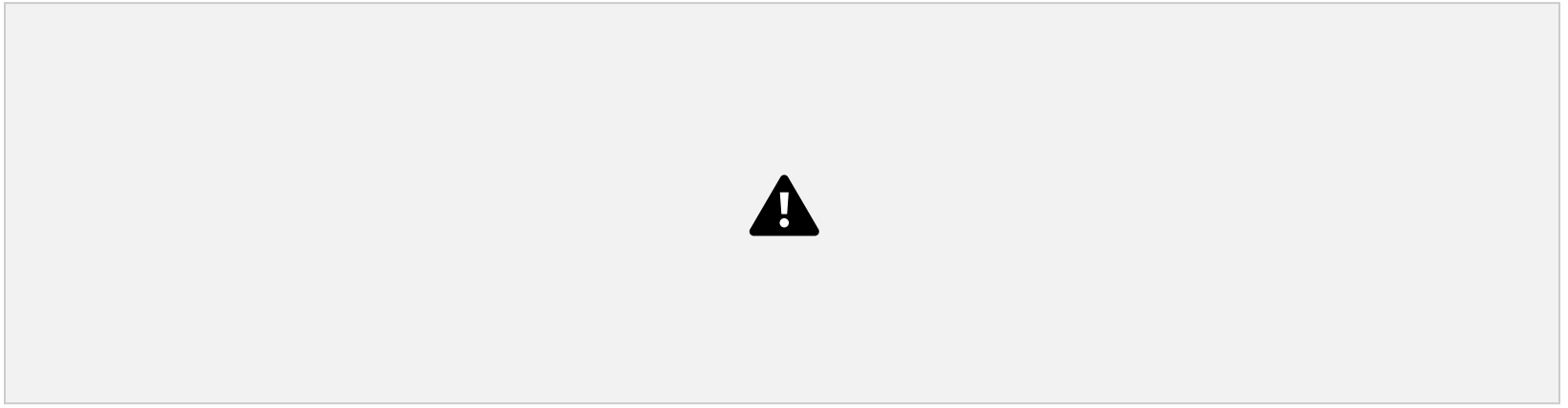
```
        args) {
            Square s= x -> {
                int r=x*x;
                System.out.println("Square : "+r);
            };
            s.findSquare(10);
        }
}
```

```
@FunctionalInterface
```

```
interface Addition{
    public int add(int x, int y);
}
public class LambdaExp {
public static void
```

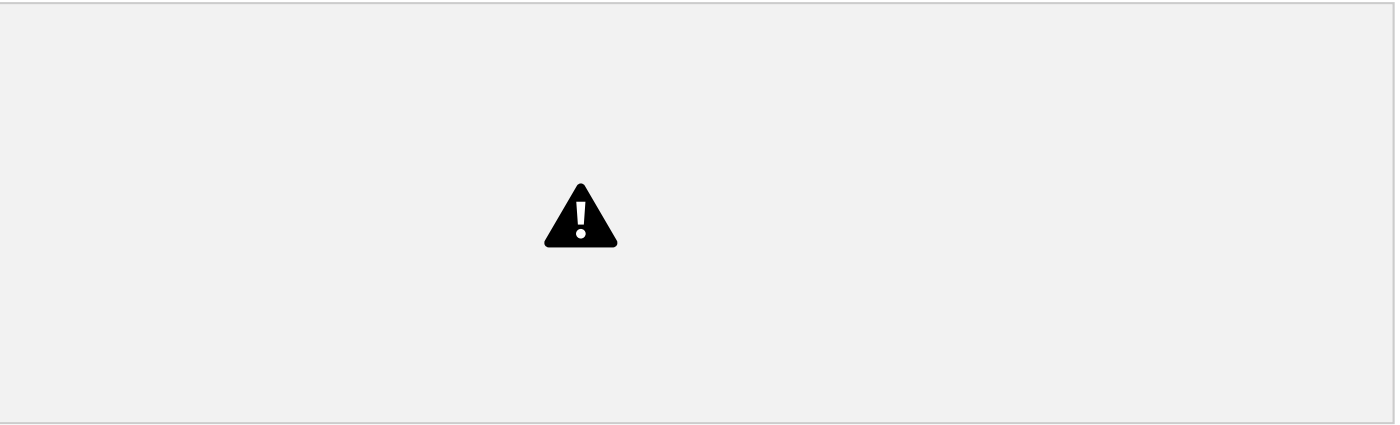
```
    main(String[] args) {
        Addition a=(x, y)->{
            return x+y;
        };
 System.out.println( “Sum : ”+a.add(10,20) );
     //also Allowed without return keyword
 Addition a=(x, y)-> x+y;
 System.out.println( “Sum : ”+a.add(10,20) ); }
}
```

- **Used to avoid the naming confliction of byte codes in a project**
- **Used to categorized the byte codes.**



Most Accessible

public: accessible all over the enclosing project.



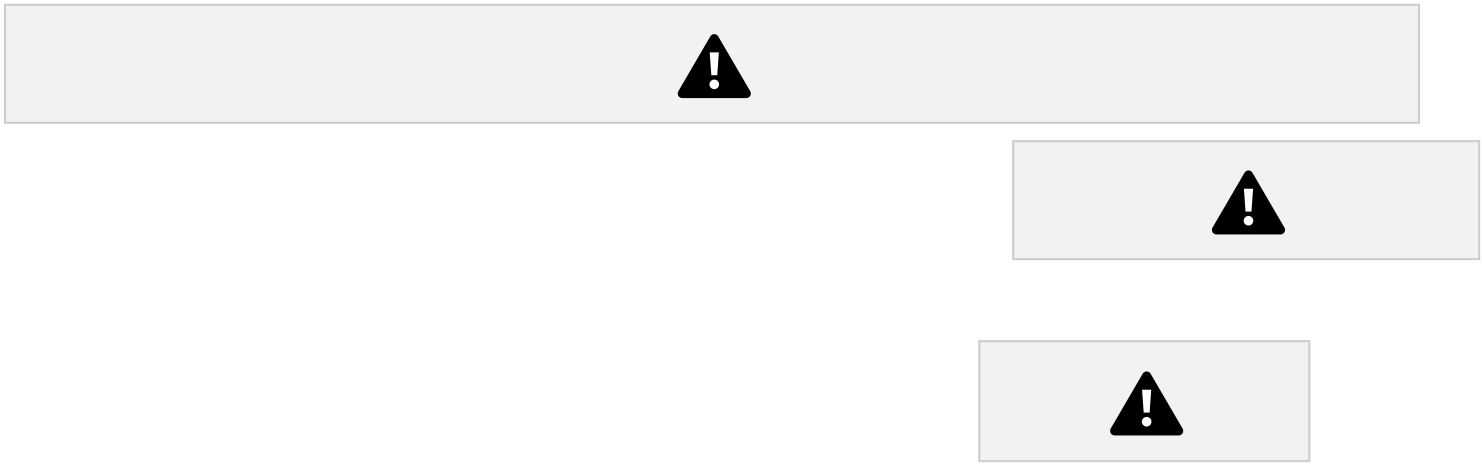
protected: accessible inside the enclosing package and inside the sub-class of any package of the enclosing project.

Default(package private): accessible only inside the

enclosing package .

private: accessible inside the enclosing class.

Less Accessible



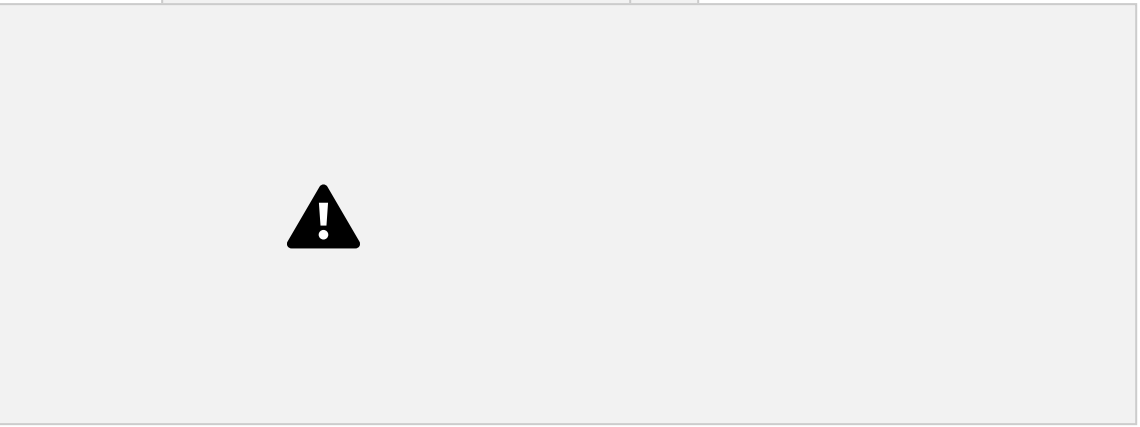
Protected member is accessible inside the enclosing package and inside the sub-class of any package of the enclosing project.

package p2

```
import p1.A;
public class B{
```

```
 public void m(){
 A a= new A();
SOP(a.x); //error  }
}


import p1.A;
public class C extends A{
public void m(){
 SOP(x);
 }
}
```

package p1

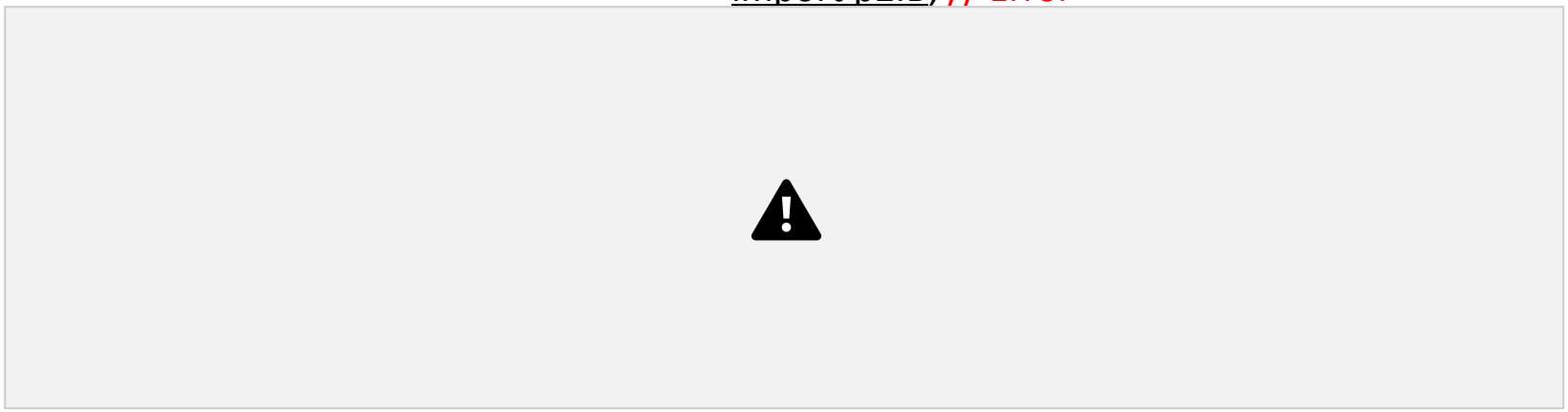


```
public class A{
 protected int x=10; }

public class B{
  public void m(){
        A a= new A();
 SOP(a.x);
 }
}

public class C extends A{
public void m(){
 SOP(x);
 }
}
```

Package inside the package known as Sub-Package or Nested Package.

package **p1**

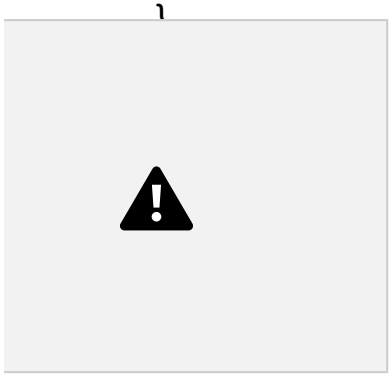
import p2.B; // Error



package **p3**

~~import p2.B~~; // Error
import p1.p2.B;
public class **D**{

public void m(){ B
b=new B(); SOP(
b.y); }
}



```
import p1.p2.B;
public class A{
 public void m(){  B
b=new B();  SOP(
b.y );  }
```

package **p1.p2**

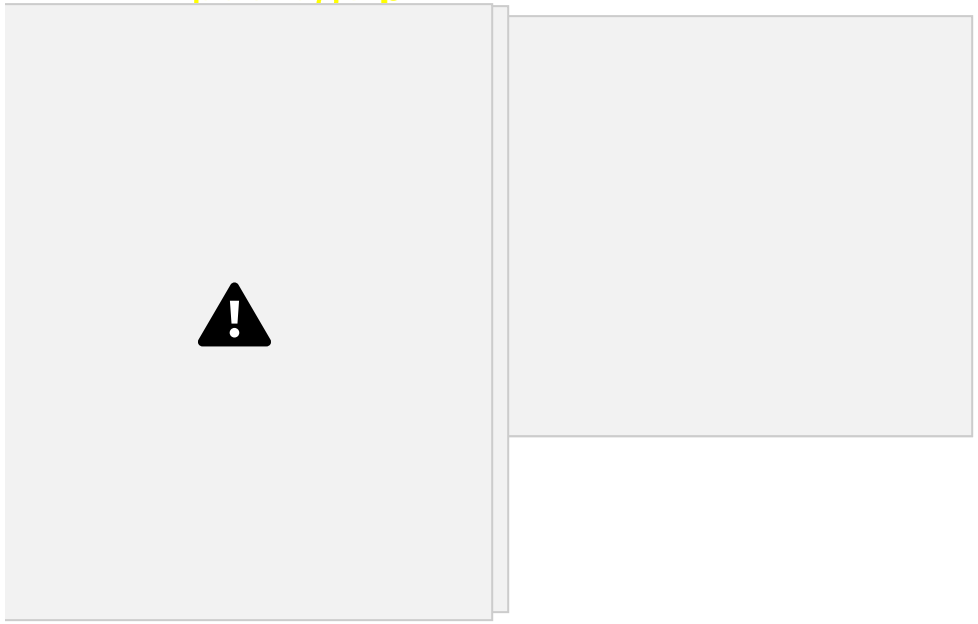
```
public class B{
public int y=20; }
```

• **import static is used to directly import static-member of the class.** • **Introduced in java 5.0**

package **p2**

```
import static p1.A.y;
import static p1.A.m2;
public class D{
 PSVM(-){
 m2();
 SOP( y );
 }
}
```

package **p1**



```
public class A{
 public int x=10;
 static public int y=20;
public void m1(){
//body
 }
 static public void m2(){
//body
 }
```

```
}
```